

Chad Galloway
CST-250 Programming in C# II
Grand Canyon University
Nov. 2, 2025
Activity 2

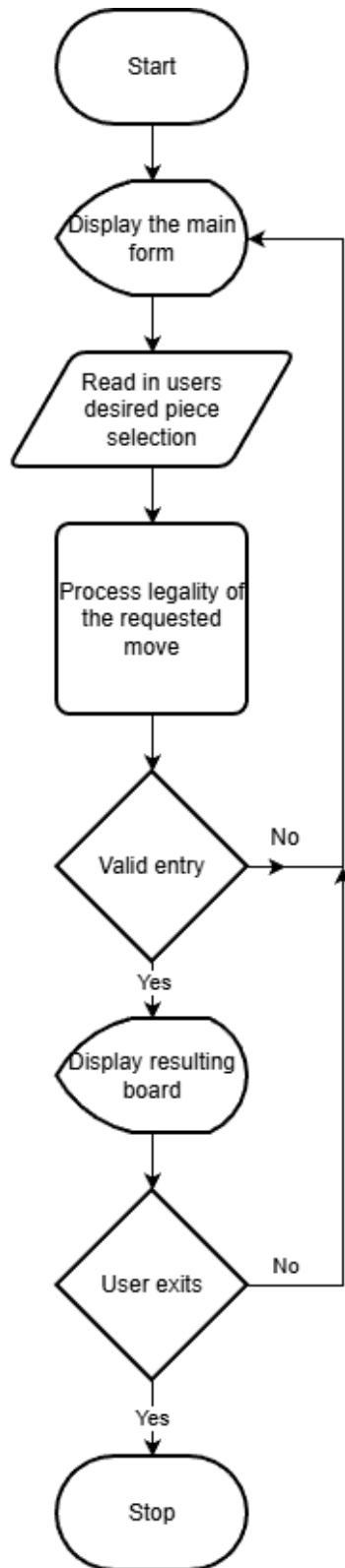
Files

<https://github.com/CGalloway3/CST-250-Projects/tree/master/Activity%202>

Video

<https://www.loom.com/share/d1423fe160ca4765b5de2ed1553af23d>

FLOW CHART



/*
* Chad Galloway
* CST - 250 Programming in C# II
* 11/25/2025
* Chess Board Class Library
* Activity 2 flowchart
* References:
*/

Figure 1: Flow chart of Activity 2

UML Class Diagram

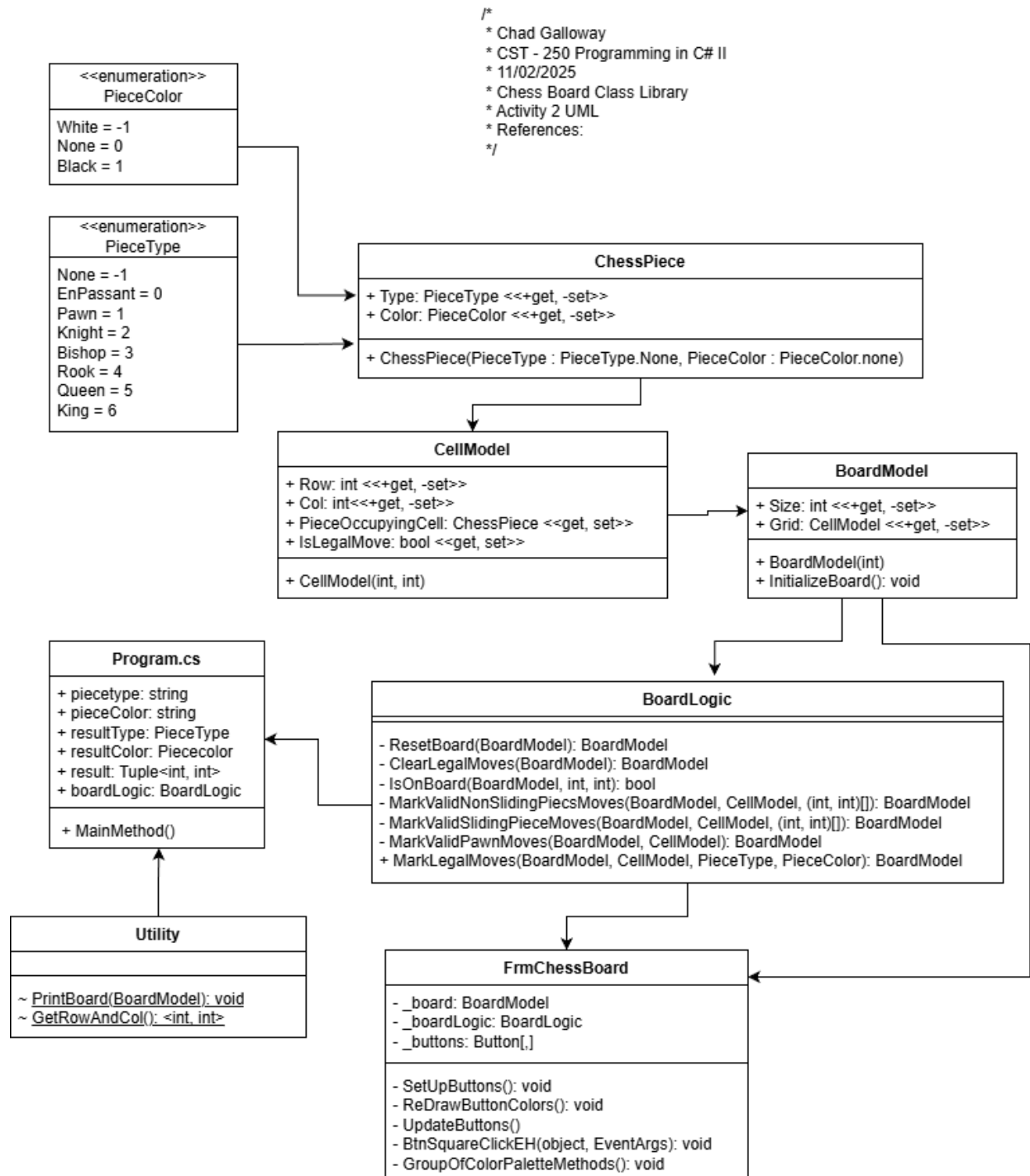
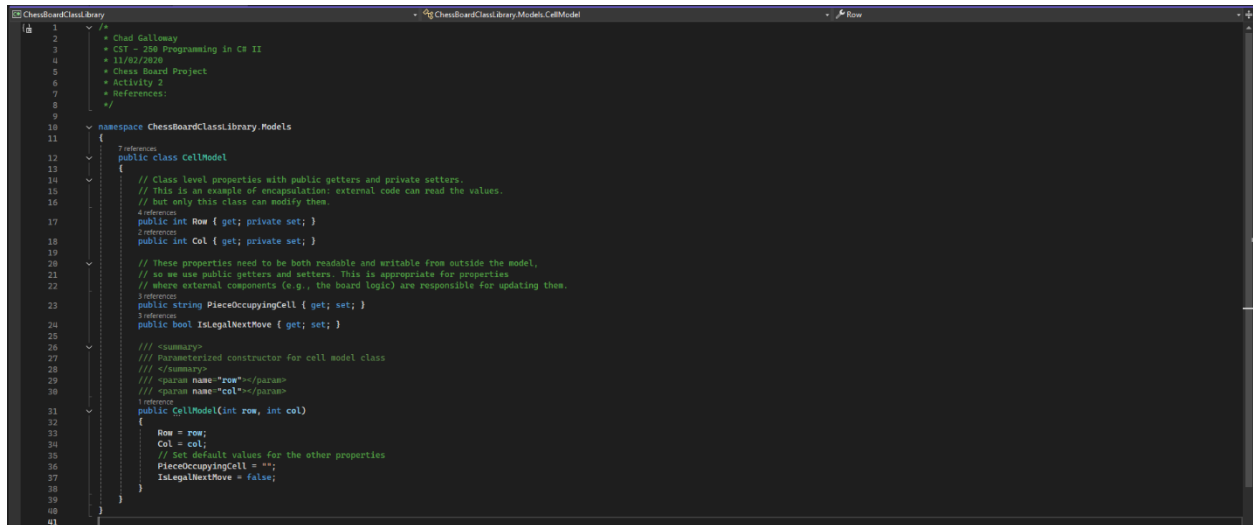


Figure 2: UML Class Diagram for Activity 2

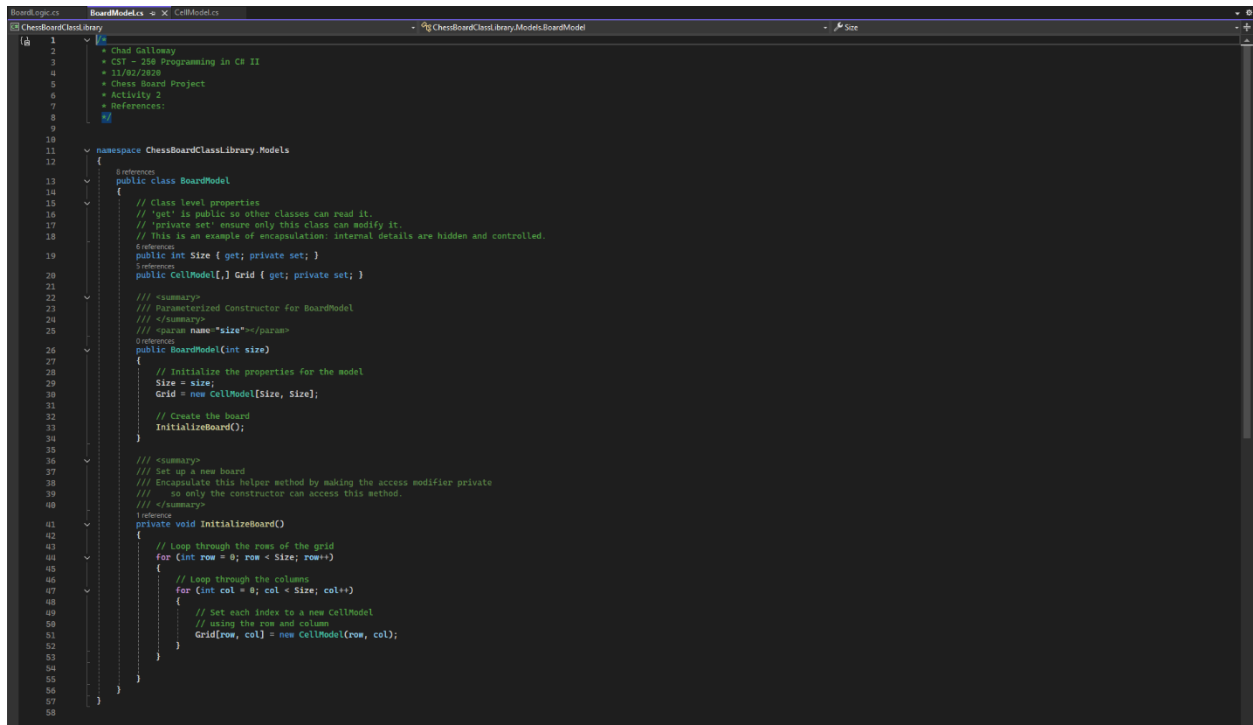
Part 1

Screen Shots



```
1 2
3  //
4  // Chad Galloway
5  // C# - 250 Programming in C# II
6  // 11/02/2020
7  // Chess Board Project
8  // Activity 2
9  // References:
10 //
11 namespace ChessBoardClassLibrary.Models
12 {
13     7 references
14     public class CellModel
15     {
16         // Class level properties with public getters and private setters.
17         // This is an example of encapsulation: external code can read the values.
18         // but only this class can modify them.
19         4 references
20         public int Row { get; private set; }
21         1 reference
22         public int Col { get; private set; }
23
24         // These properties need to be both readable and writable from outside the model.
25         // so we use public getters and setters. This is appropriate for properties
26         // where external components (e.g., the board logic) are responsible for updating them.
27         2 references
28         public string PieceOccupyingCell { get; set; }
29         3 references
30         public bool IsLegalNextMove { get; set; }
31
32         /// <summary>
33         /// Parameterized constructor for cell model class
34         /// </summary>
35         /// <param name="row">/>
36         /// <param name="col">/>
37         1 reference
38         public CellModel(int row, int col)
39         {
40             Row = row;
41             Col = col;
42             // Set default values for the other properties
43             PieceOccupyingCell = "";
44             IsLegalNextMove = false;
45         }
46     }
47 }
```

Figure 3: CellModel citations and constructor



```
BoardModels - CellModels
ChessBoardClassLibrary
1 2
3  //
4  // Chad Galloway
5  // C# - 250 Programming in C# II
6  // 11/02/2020
7  // Chess Board Project
8  // Activity 2
9  // References:
10 //
11 namespace ChessBoardClassLibrary.Models
12 {
13     6 references
14     public class BoardModel
15     {
16         // Class level properties
17         // 'get' is public so other classes can read it.
18         // 'private set' ensure only this class can modify it.
19         // This is an example of encapsulation: internal details are hidden and controlled.
20         6 references
21         public int Size { get; private set; }
22         1 reference
23         public CellModel[,] Grid { get; private set; }
24
25         /// <summary>
26         /// Parameterized Constructor for BoardModel
27         /// </summary>
28         /// <param name="size">/>
29         2 references
30         public BoardModel(int size)
31         {
32             // Initialize the properties for the model
33             Size = size;
34             Grid = new CellModel[Size, Size];
35
36             // Create the board
37             InitializeBoard();
38         }
39
40         /// <summary>
41         /// Set up a new board
42         /// Encapsulate this helper method by making the access modifier private
43         /// so only the constructor can access this method.
44         /// </summary>
45         1 reference
46         private void InitializeBoard()
47         {
48             // Loop through the rows of the grid
49             for (int row = 0; row < Size; row++)
50             {
51                 // Loop through the columns
52                 for (int col = 0; col < Size; col++)
53                 {
54                     // Set each index to a new CellModel
55                     // using the row and column
56                     Grid[row, col] = new CellModel(row, col);
57                 }
58             }
59         }
60     }
61 }
```

Figure 4: BoardModel citations and constructor

```

BoardLogic.cs | BoardModel.cs | CellModel.cs
ChessBoardClassLibrary | ChessBoardClassLibrary.Services.BusinessLogicLayer.BoardLogic | MarkLegalMoves(BoardModel board, CellModel currentCell, string chessPiece)

1  //
2  * Chad Galloway
3  * CST - 250 Programming in C# II
4  * 11/02/2020
5  * Chess Board Project
6  * Activity 2
7  * References:
8  */
9
10 using ChessBoardClassLibrary.Models;
11
12 namespace ChessBoardClassLibrary.Services.BusinessLogicLayer
13 {
14     [summary]
15     public class BoardLogic
16     {
17         /// <summary>
18         /// Reset the board by setting the
19         /// cell properties back to default
20         /// Encapsulate this method so it can only be
21         /// called from this class.
22         /// </summary>
23         /// <param name="board"></param>
24         /// <returns></returns>
25         private BoardModel ResetBoard(BoardModel board)
26         {
27             //
28             // Check if a row/column location is on the board
29             // Encapsulate this method so it can only be
30             // called from this class.
31             // </summary>
32             /// <param name="board"></param>
33             /// <param name="row"></param>
34             /// <param name="col"></param>
35             /// <returns></returns>
36             private bool IsOnBoard(BoardModel board, int row, int col)
37             {
38                 //
39                 // Mark the legal moves for the given piece and location
40                 // </summary>
41                 /// <param name="board"></param>
42                 /// <param name="currentCell"></param>
43                 /// <param name="chessPiece"></param>
44                 /// <returns></returns>
45                 public BoardModel MarkLegalMoves(BoardModel board, CellModel currentCell, string chessPiece)
46                 {
47                     // End of MarkLegalMoves method
48                 }
49             }
50         }
51     }
52 }

```

Figure 5: BoardLogic and collapsed methods with expanded summary

```

BoardLogic.cs | BoardModel.cs | CellModel.cs
ChessBoardClassLibrary | ChessBoardClassLibrary.Services.BusinessLogicLayer.BoardLogic | MarkLegalMoves(BoardModel board, CellModel currentCell, string chessPiece)

12 namespace ChessBoardClassLibrary.Services.BusinessLogicLayer
13 {
14     [summary]
15     public class BoardLogic
16     {
17         /// <summary> Reset the board by setting the cell properties back to default Enc...
18         private BoardModel ResetBoard(BoardModel board)
19         {
20             // Use a foreach loop to iterate over every cell
21             foreach (CellModel cell in board.Grid)
22             {
23                 // Set the properties to their defaults
24                 cell.IsLegalMove = false;
25                 cell.PieceOccupyingCell = "";
26             }
27             // Return the board back to the presentation layer
28             return board;
29         }
30
31         /// <summary> Check if a row/column location is on the board Encapsulate this me...
32         private bool IsOnBoard(BoardModel board, int row, int col)
33         {
34             // Get the size of the board
35             int size = board.Size;
36             // Check if the row is on the board
37             bool isRowSafe = row >= 0 && row < size;
38             // Check if the column is on the board
39             bool isColumnSafe = col >= 0 && col < size;
40             // Return true if both row and column are safe
41             return isRowSafe && isColumnSafe;
42         }
43
44         /// <summary> Mark the legal moves for the given piece and location
45         public BoardModel MarkLegalMoves(BoardModel board, CellModel currentCell, string chessPiece)
46         {
47             // Reset the board
48             board = ResetBoard(board);
49             // Use a switch statement to determine the behavior of the piece
50             switch (chessPiece.ToLower())
51             {
52                 case "knight":
53                     // Set the occupying properties for the current cell
54                     board.Grid[currentCell.Row, currentCell.Col].PieceOccupyingCell = "N";
55                     // Mark the valid moves for the knight
56                     board = MarkValidKnightMoves(board, currentCell);
57                     break;
58                 case "rook":
59                     break;
60                 case "bishop":
61                     break;
62                 case "queen":
63                     break;
64                 case "king":
65                     break;
66                 default:
67                     // If the piece is not valid, return the board as is
68                     return board;
69             }
70             // Return the updated board
71             return board;
72         }
73     }
74 }

```

Figure 6: BoardLogic: ResetBoard(), IsOnBoard(), and MarkLegalMoves() with collapsed summaries

```

100 // Return the updated board
101 return board;
102 // End of MarkLegalMoves method
103
104 /// <summary>
105 /// Mark the valid moves for the knight
106 /// Access modifier is private - meaning this method is encapsulated within the
107 /// BoardLogic class and cannot be accessed directly outside the class
108 /// </summary>
109 /// <param name="board"></param>
110 /// <param name="currentCell"></param>
111 /// <returns></returns>
112 private BoardModel MarkValidKnightMoves(BoardModel board, CellModel currentCell)
113 {
114     // Possible moves for knight row
115     int[] knightRowMoves = { 2, 1, -1, -2, -2, -1, 1, 2 };
116     // Possible moves for knight column
117     int[] knightColMoves = { 1, 2, 2, 1, -1, -2, -2, -1 };
118     // Loop through the knight moves
119     for (int i = 0; i < knightRowMoves.Length; i++)
120     {
121         // Check if each move is on the board
122         if (IsOnBoard(board, currentCell.Row + knightRowMoves[i], currentCell.Col + knightColMoves[i]))
123         {
124             // If the cell is on the board, label it as a possible move for the knight
125             board.Grid[currentCell.Row + knightRowMoves[i], currentCell.Col + knightColMoves[i]].IsLegalNextMove = true;
126         }
127     }
128     return board;
129 }

```

Figure 7: BoardLogic.MarkValidKnightMoves()

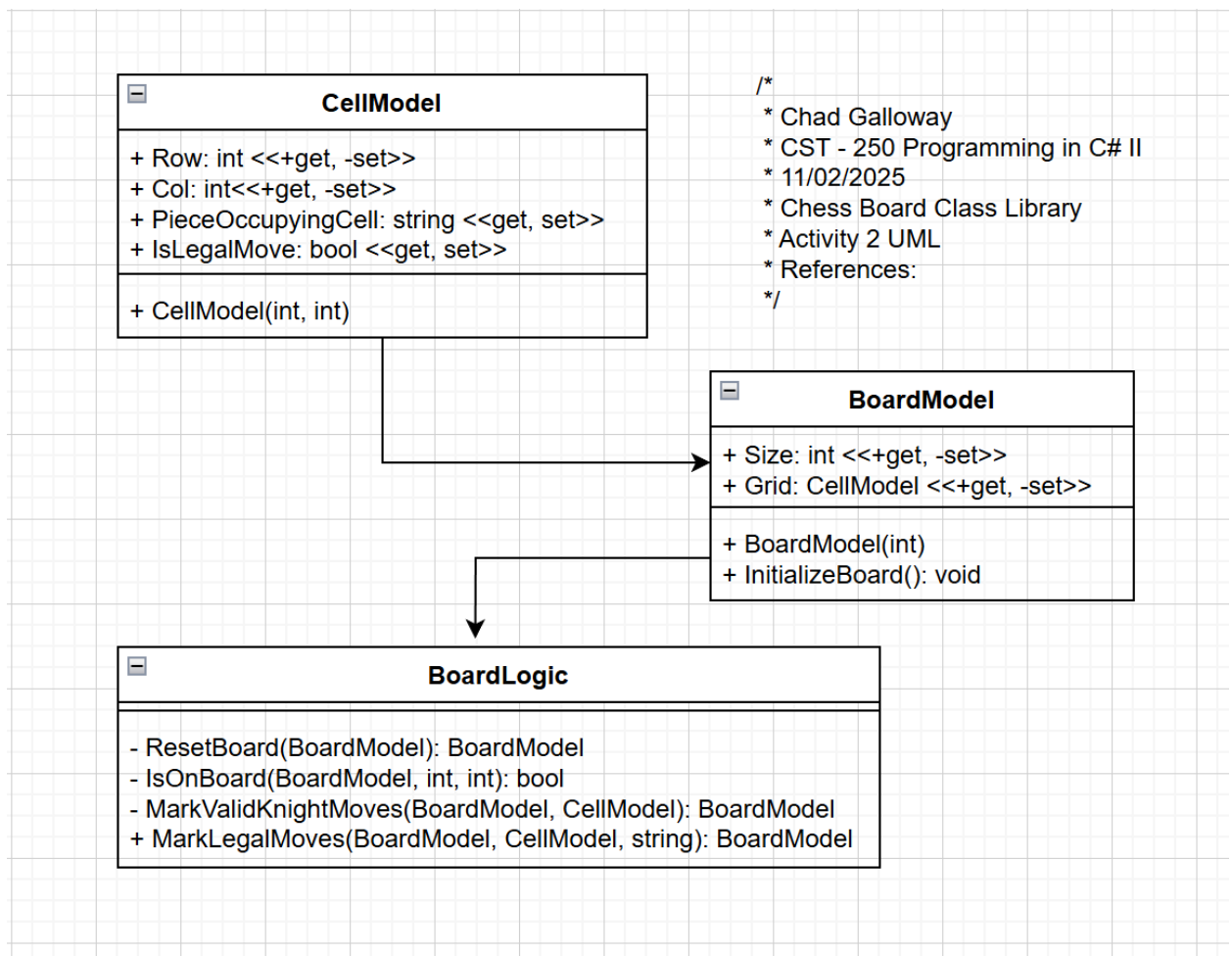
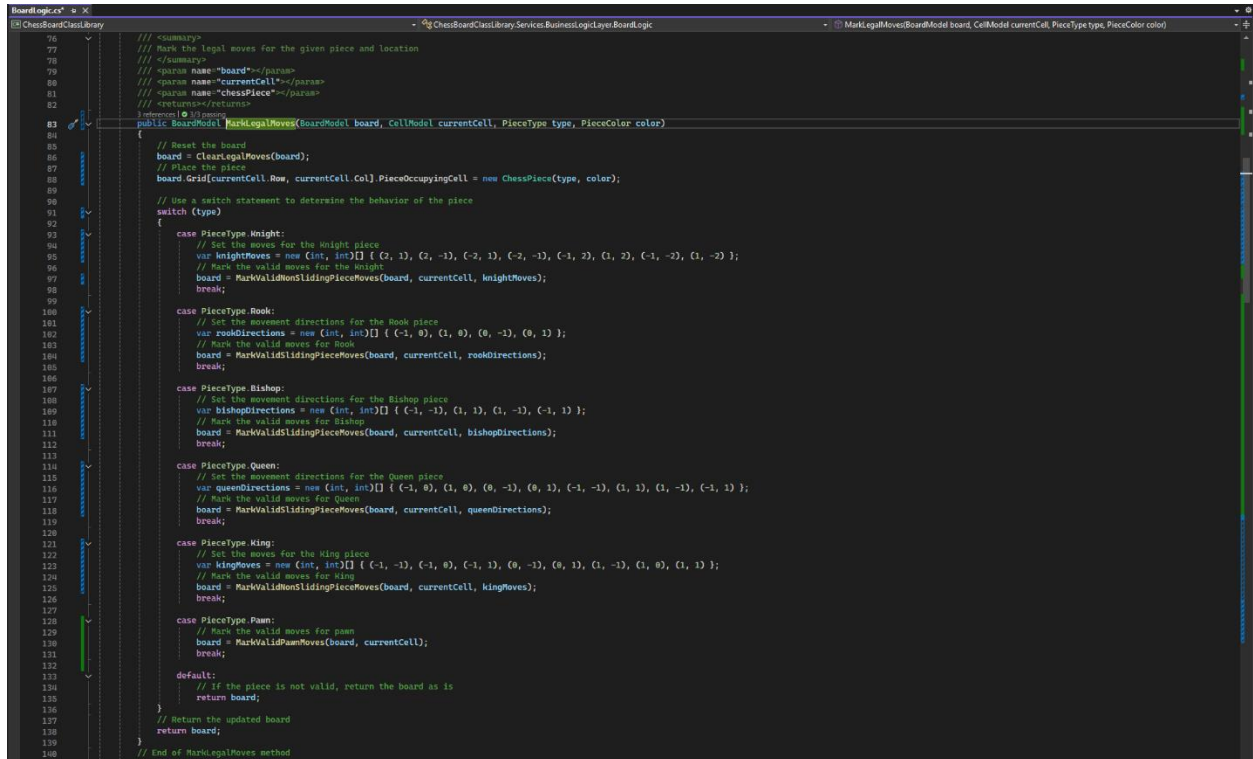


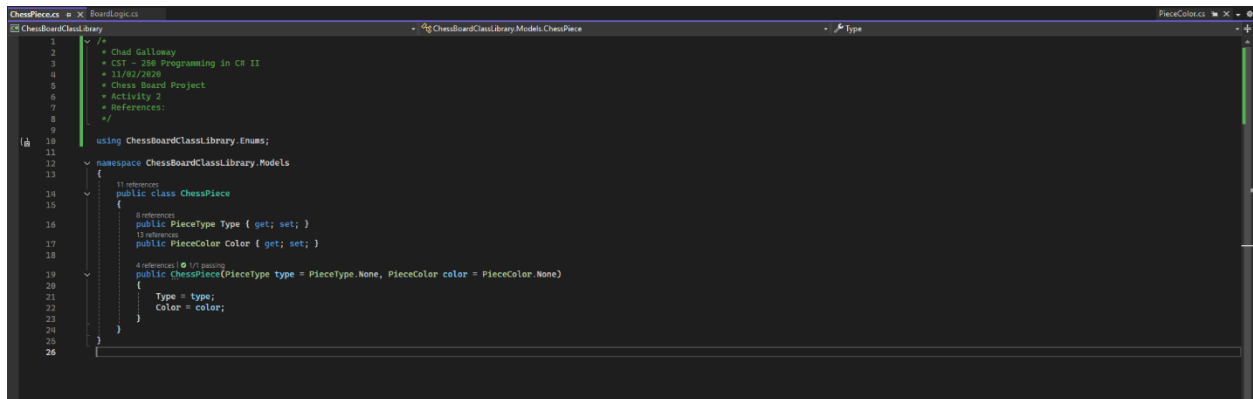
Figure 8: Screenshot of UML for CellModel, BoardModel, and BoardLogic

Challenges



```
76 // <summary>  
77 // Mark the legal moves for the given piece and location  
78 // </summary>  
79 // <param name="board">Board</param>  
80 // <param name="currentCell">CellModel</param>  
81 // <param name="chessPiece">ChessPiece</param>  
82 // <returns>Board</returns>  
83 public BoardModel MarkLegalMoves(BoardModel board, CellModel currentCell, PieceType type, PieceColor color)  
84 {  
85     // Reset the board  
86     board = ClearLegalMoves(board);  
87     // Place the piece  
88     board.Grid[currentCell.Row, currentCell.Col].PieceOccupyingCell = new ChessPiece(type, color);  
89  
90     // Use a switch statement to determine the behavior of the piece  
91     switch (type)  
92     {  
93         case PieceType.Knight:  
94             // Set the moves for the knight piece  
95             var knightMoves = new (int, int)[] { (2, 1), (2, -1), (-2, 1), (-2, -1), (-1, 2), (-1, -2), (1, 2), (1, -2) };  
96             // Mark the valid moves for the knight  
97             board = MarkValidNonSlidingPieceMoves(board, currentCell, knightMoves);  
98             break;  
99  
100        case PieceType.Rook:  
101            // Set the movement directions for the Rook piece  
102            var rookDirections = new (int, int)[] { (-1, 0), (1, 0), (0, -1), (0, 1) };  
103            // Mark the valid moves for Rook  
104            board = MarkValidSlidingPieceMoves(board, currentCell, rookDirections);  
105            break;  
106  
107        case PieceType.Bishop:  
108            // Set the movement directions for the Bishop piece  
109            var bishopDirections = new (int, int)[] { (-1, -1), (1, 1), (1, -1), (-1, 1) };  
110            // Mark the valid moves for Bishop  
111            board = MarkValidSlidingPieceMoves(board, currentCell, bishopDirections);  
112            break;  
113  
114        case PieceType.Queen:  
115            // Set the movement directions for the Queen piece  
116            var queenDirections = new (int, int)[] { (-1, 0), (1, 0), (0, -1), (0, 1), (-1, -1), (1, 1), (1, -1), (-1, 1) };  
117            // Mark the valid moves for Queen  
118            board = MarkValidSlidingPieceMoves(board, currentCell, queenDirections);  
119            break;  
120  
121        case PieceType.King:  
122            // Set the moves for the King piece  
123            var kingMoves = new (int, int)[] { (-1, -1), (-1, 0), (-1, 1), (0, -1), (0, 1), (1, -1), (1, 0), (1, 1) };  
124            // Mark the valid moves for King  
125            board = MarkValidNonSlidingPieceMoves(board, currentCell, kingMoves);  
126            break;  
127  
128        case PieceType.Pawn:  
129            // Mark the valid moves for pawn  
130            board = MarkValidPawnMoves(board, currentCell);  
131            break;  
132  
133        default:  
134            // If the piece is not valid, return the board as is  
135            return board;  
136  
137        // Return the updated board  
138        return board;  
139    }  
140    // End of MarkLegalMoves method
```

Figure 9: Updated and completed BoardLogic.MarkLegalMoves() method. Changed the string parameter for piece to type and color enums I created. Moved piece creation out of the cases and into the main body using a ChessPiece class I created.



```
1 // <summary>  
2 // Chess Piece  
3 // </summary>  
4 // <param name="type">PieceType</param>  
5 // <param name="color">PieceColor</param>  
6 // <returns>ChessPiece</returns>  
7  
8  
9  
10 using ChessBoardClassLibrary.Enums;  
11  
12 namespace ChessBoardClassLibrary.Models  
13 {  
14     // 11 references  
15     public class ChessPiece  
16     {  
17         // 8 references  
18         public PieceType Type { get; set; }  
19         // 13 references  
20         public PieceColor Color { get; set; }  
21  
22         // 4 references (0 / 1 passing)  
23         public ChessPiece(PieceType type = PieceType.None, PieceColor color = PieceColor.None)  
24         {  
25             Type = type;  
26             Color = color;  
27         }  
28     }  
29 }  
30
```

Figure 10: ChessPiece class. Contains piece type and piece color. Currently the piece doesn't do anything (no methods). In the future it might have some movement methods added in we will see.

```

1  //
2  //
3  //
4  //
5  //
6  //
7  //
8  //
9  //
10 namespace ChessBoardClassLibrary.Enums
11 {
12     /// <summary>
13     /// Represents the types of chess pieces.
14     /// PieceType None (-1) represents an empty square.
15     /// PieceType EnPassant (0) is a temporary "ghost" piece,
16     /// used exclusively to indicate en passant availability to pawns;
17     /// other pieces treat it as empty.
18     /// </summary>
19     /// <remarks>
20     public enum PieceType
21     {
22         None = -1,
23         EnPassant = 0,
24         Pawn = 1,
25         Knight = 2,
26         Bishop = 3,
27         Rook = 4,
28         Queen = 5,
29         King = 6
30     }
31 }

```

Figure 10a: PieceType enum to hold all the different piece types including none and en passant a "ghost" piece type only visible in code to pawns for preforming the en passant move.

```

1  //
2  //
3  //
4  //
5  //
6  //
7  //
8  //
9  //
10 namespace ChessBoardClassLibrary.Enums
11 {
12     /// <summary>
13     /// Signifies a piece color for easy comparison and
14     /// numeric values are signed to simplify pawn movement along the row axis:
15     /// - White = -1 - moves up (decreasing row)
16     /// - Black = 1 - moves down (increasing row)
17     /// - None = 0 - signifies empty or neutral squares
18     /// </summary>
19     /// <remarks>
20     public enum PieceColor
21     {
22         White = -1,
23         None = 0,
24         Black = 1
25     }
26 }

```

Figure 10b: PieceColor emun to hold the colors of pieces. Used to determine if a legal move of taking an enemy piece is available and it is also a directional indicator of the movement directions of the two colored pieces for pawns. Black pawns move down the board +1, and white pawns move up the board -1.


```

BoardLogic.cs
ChessBoardClassLibrary
ChessBoardClassLibrary.Services.BusinessLogicLayer.BoardLogic
MarkValidPawnMoves(BoardModel board, CellModel currentCell)

256 /// <summary> Marks valid moves for a pawn
257 /// Access modifier is private - meaning this method is encapsulated within the
258 /// BoardLogic class and cannot be accessed directly outside the class.
259 /// </summary>
260 /// <param name="board">The current game board model.</param>
261 /// <param name="currentCell">The cell containing the active piece we are testing.</param>
262 /// <returns>The updated board with legal move flags applied. No Pieces are moved or altered.</returns>
263 /// </summary>
264 /// This method assumes that the black pieces start at the top of the board
265 /// and the white pieces start at the bottom. If that is not the case in your UI
266 /// then the pawn moves are coded incorrectly here. Reverse the white and black values
267 /// in <see cref="PieceColor"/> to rectify this and make white the top color.
268 /// Note: PieceColor enum values are signed to represent board movement direction.
269 /// White = -1 moves "up" the board, Black = 1 moves "down".
270 /// This method enforces standard pawn movement rules, including:
271 /// - Forward single-step moves if unoccupied
272 /// - Double-step opening move from the start rank
273 /// - Diagonal captures of opposing pieces
274 /// En passant and promotion are handled elsewhere in the move logic.
275 /// </summary>
276
277 private BoardModel MarkValidPawnMoves(BoardModel board, CellModel currentCell)
278 {
279 }
280 }

```

Figure 13: Summary for MarkValidPawnMoves()

```

BoardLogic.cs
ChessBoardClassLibrary
ChessBoardClassLibrary.Services.BusinessLogicLayer.BoardLogic
MarkValidPawnMoves(BoardModel board, CellModel currentCell)

256 /// <summary> Marks valid moves for a pawn Access modifier is private - meaning ...
257
258 private BoardModel MarkValidPawnMoves(BoardModel board, CellModel currentCell)
259 {
260     // Declare and initialize
261     ChessPiece currentPiece = currentCell.PieceOccupyingCell;
262
263     // Handle single move ahead
264     // Calculate the coordinates of one move straight ahead and set the target variables
265     // We don't need to check for IsOnBoard because any piece that reaches the edge is converted to a queen
266     int targetRow = currentCell.Row + ((int)currentPiece.Color);
267     int targetCol = currentCell.Col;
268     CellModel targetCell = board.Grid[targetRow, targetCol];
269     ChessPiece targetPiece = targetCell.PieceOccupyingCell;
270
271     // Determine if the move one space straight ahead is clear
272     if (targetPiece.Type == PieceType.None)
273     {
274         // Move is clear mark it as a legal move
275         targetCell.IsLegalNextMove = true;
276     }
277
278     // Handle double move from start position
279
280     // Determine if a double move is valid given all the cases
281     if (!IsOnBoard(board, targetRow + ((int)currentPiece.Color, targetCol)) // case 1 cell we are checking is actually on the board.
282         && targetCell.IsLegalNextMove // case 2 can we "slide" past this spot because it is a legal move
283         && currentCell.Row == 1 // case 3 ensure pawn is on its start rank; IsOnBoard ensures this is for the correct side.
284         && board.Grid[targetRow + ((int)currentPiece.Color, targetCol).PieceOccupyingCell.Type == PieceType.None) // case 4 the spot where we want to end up is empty also
285     {
286         // Spot is empty and clear to slide in from our start row
287         board.Grid[targetRow + ((int)currentPiece.Color, targetCol).IsLegalNextMove = true;
288     }
289
290     // Handle diagonal capture
291     // Set capture moves to a variable for pawns
292     var pawnCaptures = new (int, int)[] { ((int)currentPiece.Color, 1), ((int)currentPiece.Color, -1) };
293     // Iterate over capture moves
294     foreach (var (rowOffset, colOffset) in pawnCaptures)
295     {
296         // Calculate the coordinates of the possible capture
297         targetRow = currentCell.Row + rowOffset;
298         targetCol = currentCell.Col + colOffset;
299
300         // Determine if the cell is on the board
301         if (!IsOnBoard(board, targetRow, targetCol))
302         {
303             continue; // Cell is off the board, continue to the next move location
304         }
305
306         // we are on the board set target variable values
307         targetCell = board.Grid[targetRow, targetCol];
308         targetPiece = targetCell.PieceOccupyingCell;
309
310         // Determine if a piece is present and it is an enemy
311         if (targetPiece.Type != PieceType.None && targetPiece.Color != currentPiece.Color)
312         {
313             // There is an enemy piece available for capture
314             targetCell.IsLegalNextMove = true;
315         }
316     }
317
318     return board;
319 }
320 }

```

Figure 14: BoardLogic.MarkValidPawnMoves() marks all the movement cases for pawns. Promotions for reaching the last rank and en passant will be handled in the actual move logic. Here we are just highlighting what might be a legal move.

```
UnitTests - X
ChessBoardClassLibrary.Tests
  • ChessBoardClassLibrary.Tests\UnitTest1
  • MarkValidKingMoves_MarkProperMoves()

1  // *
2  // * Chad Gallowsay
3  // * CST - 250 Programming in C# II
4  // * 11/02/2020
5  // * Chess Board Project
6  // * Activity 2
7  // * References:
8  // *
9
10 using ChessBoardClassLibrary.Models;
11 using ChessBoardClassLibrary.Services.BusinessLogicLayer;
12
13 namespace ChessBoardClassLibrary.Tests
14 {
15     [Fact]
16     public class UnitTest1
17     {
18         [Fact]
19         [12 references]
20         public void MarkValidKnightMoves_MarkProperMoves()
21         {
22             // Arrange
23             BoardLogic boardLogic = new BoardLogic();
24             BoardModel board = new BoardModel(8);
25
26             // Act
27             board = boardLogic.MarkLegalMoves(board, board.Grid[2, 7], Enums.PieceType.Knight, Enums.PieceColor.White);
28
29             // Assert
30             Assert.NotNull(board);
31             Assert.True(board.Grid[4, 6].IsLegalNextMove);
32             Assert.True(board.Grid[0, 6].IsLegalNextMove);
33         }
34
35         [Fact]
36         [10 references]
37         public void MarkValidRookMoves_MarkProperMoves()
38         {
39             // Arrange
40             BoardLogic boardLogic = new BoardLogic(); //Initiate the logic
41             BoardModel board = new BoardModel(8); // Initialize the board as blank
42             board.Grid[7, 2].PieceOccupyingCell = new ChessPiece(Enums.PieceType.Pawn, Enums.PieceColor.Black); // add a piece of the opposite color in the way
43
44             // Act: Call board logic mark legal moves on the board for a rook at 7,5
45             board = boardLogic.MarkLegalMoves(board, board.Grid[7, 5], Enums.PieceType.Rook, Enums.PieceColor.White);
46
47             // Assert
48             Assert.NotNull(board);
49             Assert.True(board.Grid[5, 5].IsLegalNextMove);
50             Assert.True(board.Grid[7, 2].IsLegalNextMove);
51             Assert.False(board.Grid[7, 1].IsLegalNextMove);
52         }
53
54         [Fact]
55         [10 references]
56         public void MarkValidKingMoves_MarkProperMoves()
57         {
58             // Arrange
59             BoardLogic boardLogic = new BoardLogic();
60             BoardModel board = new BoardModel(8);
61
62             // Act
63             board = boardLogic.MarkLegalMoves(board, board.Grid[4, 7], Enums.PieceType.King, Enums.PieceColor.White);
64
65             // Assert
66             Assert.NotNull(board);
67             Assert.True(board.Grid[5, 6].IsLegalNextMove);
68             Assert.False(board.Grid[4, 6].IsLegalNextMove);
69         }
70     }
71 }
```

Figure 15: Several unit tests were created for the project to make coding the different MarkValid methods easier.

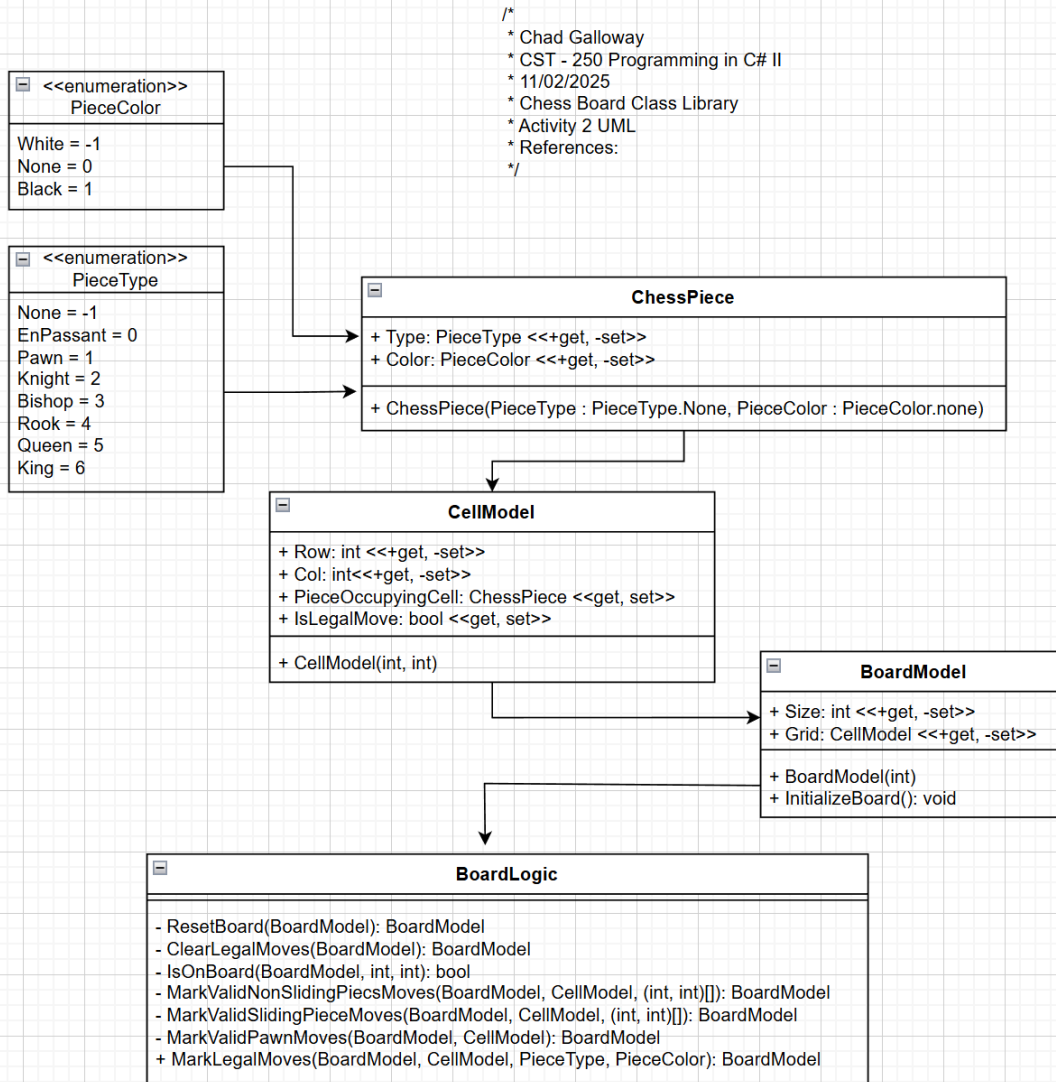
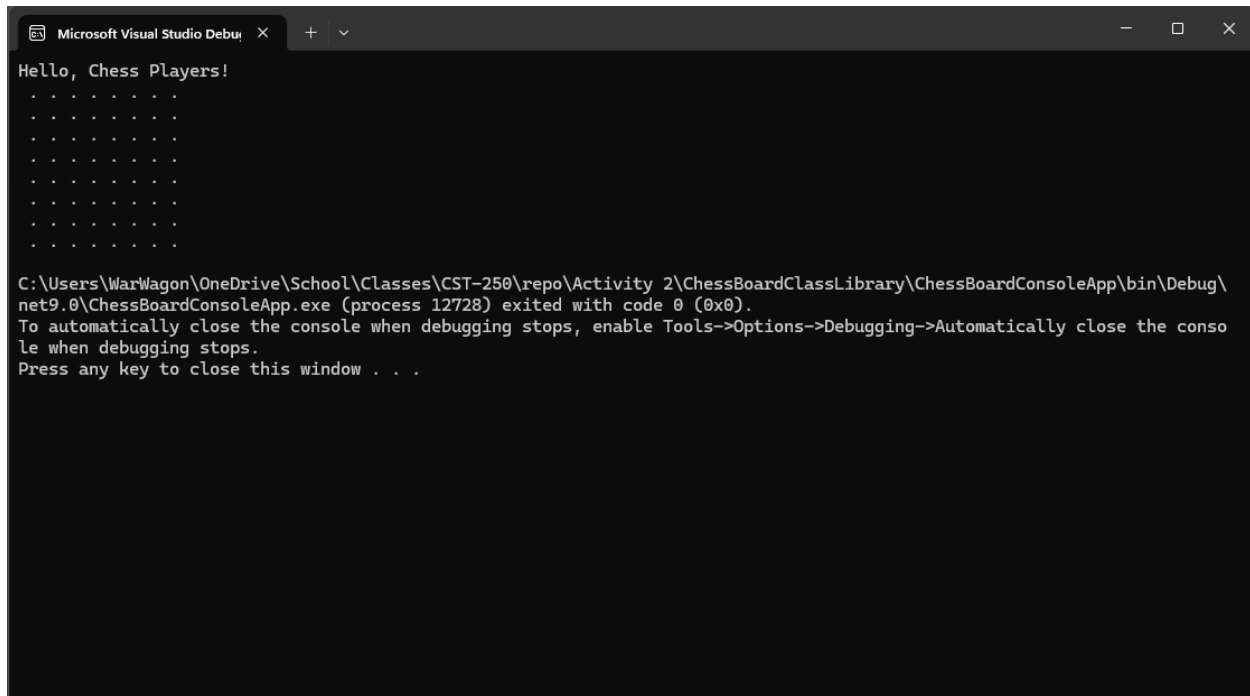


Figure 16: Projects UML so far.

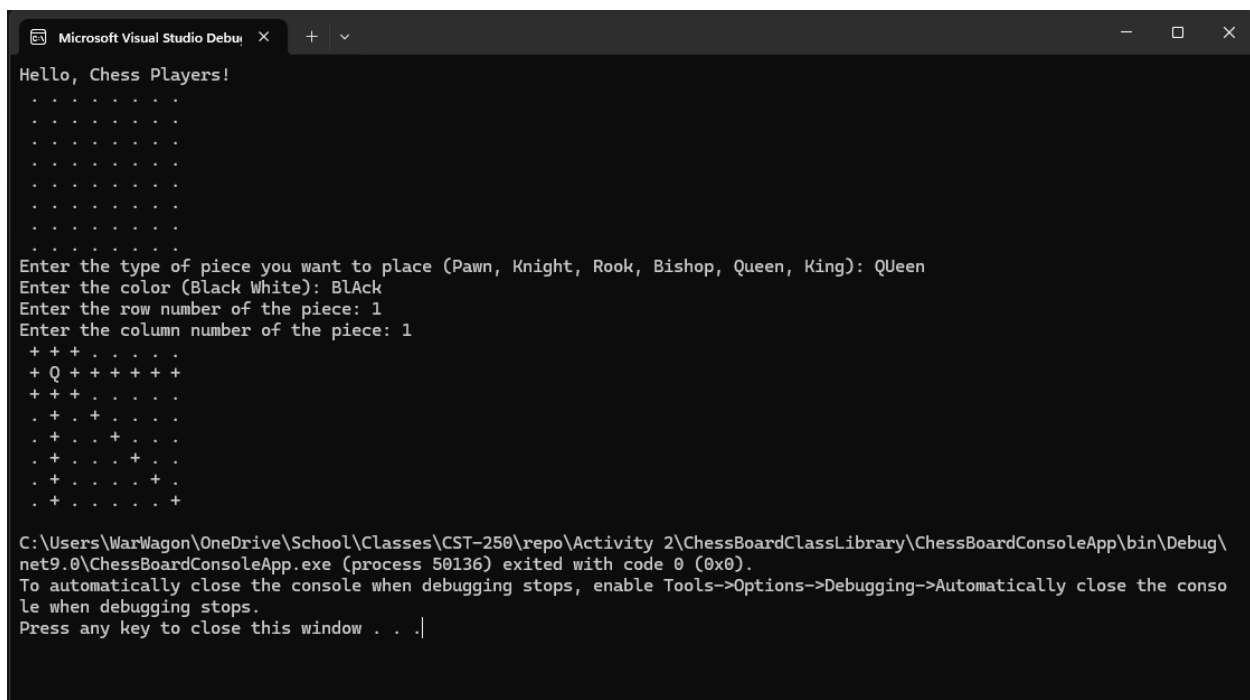
Part 2 Screen Shots



```
Microsoft Visual Studio Debug Console
Hello, Chess Players!
. . . . .
. . . . .
. . . . .
. . . . .
. . . . .
. . . . .
. . . . .
. . . . .

C:\Users\WarWagon\OneDrive\School\Classes\CST-250\repo\Activity 2\ChessBoardClassLibrary\ChessBoardConsoleApp\bin\Debug\net9.0\ChessBoardConsoleApp.exe (process 12728) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the console when debugging stops.
Press any key to close this window . . .
```

Figure 17: Part 2 App Running after writing PrintBoard() method



```
Microsoft Visual Studio Debug Console
Hello, Chess Players!
. . . . .
. . . . .
. . . . .
. . . . .
. . . . .
. . . . .
. . . . .
. . . . .

Enter the type of piece you want to place (Pawn, Knight, Rook, Bishop, Queen, King): Queen
Enter the color (Black White): Black
Enter the row number of the piece: 1
Enter the column number of the piece: 1
+ + + . . . .
+ Q + + + + +
+ + + . . . .
. + + . . . .
. + . + . . .
. + . . + . .
. + . . . + .
. + . . . . +
. + . . . . +

C:\Users\WarWagon\OneDrive\School\Classes\CST-250\repo\Activity 2\ChessBoardClassLibrary\ChessBoardConsoleApp\bin\Debug\net9.0\ChessBoardConsoleApp.exe (process 50136) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the console when debugging stops.
Press any key to close this window . . .
```

Figure 18: Screenshot of successful output with a Queen at 1, 1

```

Program.cs - X
ChessBoardConsoleApp - Utility
PrintBoard(BoardModel board)

1  //
2  // Chad Galloway
3  // CST - 280 Programming in C# II
4  // 11/02/2020
5  // Chess Board Project
6  // Activity 2
7  // References:
8  //
9  //
10 using ChessBoardClassLibrary.Enums;
11 using ChessBoardClassLibrary.Models;
12 using ChessBoardClassLibrary.Services.BusinessLogicLayer;
13 using System.Xml.XPath;
14
15 //-----
16 // Start of Main Method
17 //-----
18
19 // Declare and Initialize
20 string pieceType = "";
21 string pieceColor = "";
22 PieceType resultType = PieceType.None;
23 PieceColor resultColor = PieceColor.None;
24 Tuple<int, int> result;
25 BoardLogic boardLogic = new BoardLogic();
26
27 // Print a welcome message for the user
28 Console.WriteLine("Hello, Chess Players!");
29
30 // Create a new chess board
31 BoardModel board = new BoardModel();
32
33 // Show the empty board
34 Utility.PrintBoard(board);
35
36 // Prompt the user for the type of chess piece
37 Console.WriteLine("Enter the type of piece you want to place (Pawn, Knight, Rook, Bishop, Queen, King): ");
38 pieceType = Console.ReadLine();
39 Console.WriteLine("Enter the color (Black/White): ");
40 pieceColor = Console.ReadLine();
41 resultType = PieceTypeConverter.ConvertStringToPieceType(pieceType);
42 resultColor = PieceColorConverter.ConvertStringToPieceColor(pieceColor);
43
44 // Prompt the user for the location of the chess piece
45 result = Utility.GetRowAndCol();
46
47 // Mark the legal moves based on the input
48 board = boardLogic.MarkLegalMoves(board, board.Grid[result.Item1, result.Item2], resultType, resultColor);
49
50 // Print out the new chess board
51 Utility.PrintBoard(board);
52
53 //-----
54 // End of Main Method
55 //-----
56
57 // Define a utility class
58 //-----
59 //-----
60 3 references
61 public static class Utility
117

```

Figure 19: Screen shot of the Program.cs and its' citations

```

Program.cs - X
ChessBoardConsoleApp - Utility
PrintBoard(BoardModel board)

62 /// <summary>
63 /// Print the given board to the console
64 /// </summary>
65 /// <param name="board">BoardModel</param>
66 /// <returns>
67 2 references
68 internal static void PrintBoard(BoardModel board)
69 {
70     // Loop over the rows of the board
71     for (int row = 0; row < board.Size; row++)
72     {
73         // Loop over the columns of the board
74         for (int col = 0; col < board.Size; col++)
75         {
76             // Get the current cell from the grid
77             CellModel cell = board.Grid[row, col];
78             // Check if the current cell is a legal move
79             if (cell.IsLegalNextMove)
80             {
81                 // Print a . for a legal move
82                 Console.WriteLine(" ");
83             }
84             // Check if there is a piece occupying the cell
85             else if (cell.PieceOccupyingCell.SignifyingLetter != "")
86             {
87                 // Print the chess piece letter
88                 Console.WriteLine($" {cell.PieceOccupyingCell.SignifyingLetter}");
89             }
90             else
91             {
92                 // Print a . for anything else
93                 Console.WriteLine(" ");
94             }
95         }
96         // Start a new line after every row
97         Console.WriteLine();
98     }
99 } // End of PrintBoard method
100

```

Figure 20: PrintBoard() method

```

Program.cs - X
ChessBoardConsoleApp - Utility
GetRowAndCol()

99 /// <summary>
100 /// Get the row and column for the piece
101 /// </summary>
102 /// <returns>
103 1 reference
104 internal static Tuple<int, int> GetRowAndCol()
105 {
106     // Get the row from the user
107     Console.WriteLine("Enter the row number of the piece: ");
108     int row = int.Parse(Console.ReadLine());
109
110     // Get the column from the user
111     Console.WriteLine("Enter the column number of the piece: ");
112     int col = int.Parse(Console.ReadLine());
113
114     // Return the data
115     return Tuple.Create(row, col);
116 }
117

```

Figure 21: GetRowAndCol() method

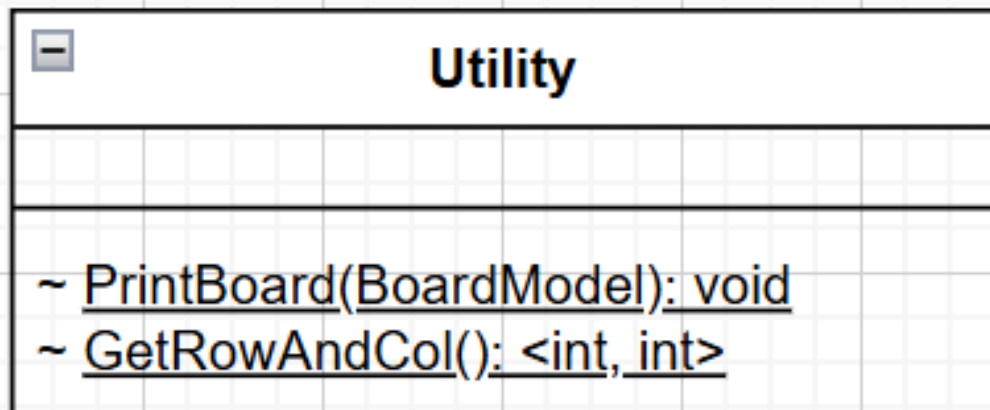


Figure 22: UML

Challenges

```
Microsoft Visual Studio Debug Console
Hello, Chess Players!
  0 1 2 3 4 5 6 7
+---+
0 | . | . | . | . | . | . | . |
+---+
1 | . | . | . | . | . | . | . |
+---+
2 | . | . | . | . | . | . | . |
+---+
3 | . | . | . | . | . | . | . |
+---+
4 | . | . | . | . | . | . | . |
+---+
5 | . | . | . | . | . | . | . |
+---+
6 | . | . | . | . | . | . | . |
+---+
7 | . | . | . | . | . | . | . |
+---+
  0 1 2 3 4 5 6 7
+---+
0 | . | . | . | . | . | . | . |
+---+
1 | Q | . | . | . | . | . | . |
+---+
2 | . | . | . | . | . | R | . | N |
+---+
3 | . | . | . | . | . | . | . | . |
+---+
4 | . | . | . | . | . | . | . | . |
+---+
5 | . | . | K | . | . | . | P | . |
+---+
6 | . | . | . | B | . | . | . | . |
+---+
7 | . | . | . | . | . | . | . | . |
+---+
Enter the type of piece you want to place (Pawn, Knight, Rook, Bishop, Queen, or King): Queen
Enter the color (Black or White): White
Enter the row number of the piece: 2
Enter the column number of the piece: 2
  0 1 2 3 4 5 6 7
+---+
0 | + | . | + | . | + | . | . |
+---+
1 | Q | + | + | + | . | . | . |
+---+
2 | + | + | Q | + | + | + | . | N |
+---+
3 | . | + | + | + | . | . | . |
+---+
4 | + | . | + | . | + | . | . |
+---+
5 | . | . | + | . | . | . | P | . |
+---+
6 | . | . | . | B | . | . | + | . |
+---+
7 | . | . | . | . | . | . | . | + |
+---+
C:\Users\WarWagon\OneDrive\School\Classes\CST-250\repo\Activity 2\ChessBoardClassLibrary\ChessBoardConsoleApp\bin\Debug\net9.0\ChessBoardConsoleApp.exe (process 52700) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the console when debugging stops.
Press any key to close this window . . .
```

Figure 23: Console running after successful completion of the challenge tasks. The First board is default. The second board has some pieces added for demonstration and the third board shows off the capabilities of the MarkValid methods I created. White + is a possible move and a red + is a possible piece capture opportunity. With further moves behind pieces that are present being unavailable. Changing the Queen to black removes the red + and displays the piece type

because the rook and the king displayed on board 2 are black. Further improvements would be changing the pieces display colors.... I just thought of a novel way to do it so see below.

```
Microsoft Visual Studio Debug Console
7 | . | . | . | . | . | . | . |
+---+
0 | . | . | . | . | . | . | . |
+---+
1 | Q | . | . | . | . | . | . |
+---+
2 | . | . | . | . | . | R | . | N |
+---+
3 | . | . | . | . | . | . | . |
+---+
4 | . | . | . | . | . | . | . |
+---+
5 | . | . | K | . | . | . | P |
+---+
6 | . | . | . | B | . | . | . |
+---+
7 | . | . | . | . | . | . | . |

Enter the type of piece you want to place (Pawn, Knight, Rook, Bishop, Queen, or King): Queen
Enter the color (Black or White): White
Enter the row number of the piece: 2
Enter the column number of the piece: 2
0 | + | . | + | . | + | . | . |
+---+
1 | Q | + | + | + | . | . | . |
+---+
2 | + | + | Q | + | + | + | . | N |
+---+
3 | . | + | + | + | . | . | . |
+---+
4 | + | . | + | . | + | . | . |
+---+
5 | . | . | + | . | . | + | P |
+---+
6 | . | . | . | B | . | . | + |
+---+
7 | . | . | . | . | . | . | + |

C:\Users\WarWagon\OneDrive\School\Classes\CST-250\repo\Activity 2\ChessBoardClassLibrary\ChessBoardConsoleApp\bin\Debug\net9.0\ChessBoardConsoleApp.exe (process 11644) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the console when debugging stops.
Press any key to close this window . . .
```

Figure 24: Screen shot of the application with colored pieces implemented. Some other color choices may be in order later like making valid moves yellow and attempting to properly color the board checkered black and white but those are lofty and a bit much for a simple console version when we are about to make a windows forms version with a much better GUI.

```

92  // Summary
93  // Print the given board to the console
94  // Summary
95  // Param name="board" Param
96  internal static void PrintBoard(BoardModel board)
97  {
98  // Print the header row (column numbers)
99  Console.WriteLine(" "); // Spacer for the row index column
100 for (int col = 0; col < board.Size; col++)
101 Console.WriteLine();
102
103 // Print the top border
104 Console.WriteLine(" ");
105 for (int col = 0; col < board.Size; col++)
106 Console.WriteLine();
107
108 // Loop over the rows of the board
109 for (int row = 0; row < board.Size; row++)
110 {
111 // Print the row index
112 Console.WriteLine($" {row} |");
113
114 // Loop over the columns of the board
115 for (int col = 0; col < board.Size; col++)
116 {
117 // Get the current cell from the grid
118 CellModel cell = board.Grid[row, col];
119 // Check if the current cell is a legal move
120 if (cell.IsLegalNextMove)
121 {
122 // Print a + for a legal move
123 // If the move is an over take of an enemy piece mark it as red.
124 if (cell.PieceOccupyingCell.Type != PieceType.None)
125 {
126 Console.ForegroundColor = ConsoleColor.Red;
127 Console.WriteLine(" + ");
128 Console.ResetColor();
129 }
130 // Check if there is a piece occupying the cell
131 else if (cell.PieceOccupyingCell.SignifyingLetter != "")
132 {
133 // Print the chess piece letter
134 if (cell.PieceOccupyingCell.Color == PieceColor.White)
135 {
136 Console.ForegroundColor = ConsoleColor.Black;
137 Console.BackgroundColor = ConsoleColor.White;
138 }
139 Console.WriteLine($" {cell.PieceOccupyingCell.SignifyingLetter} ");
140 Console.ResetColor();
141 }
142 else
143 {
144 // Print a . for anything else
145 Console.WriteLine(" . ");
146 }
147 Console.WriteLine("|");
148 }
149 }
150 // Start a new line after every row
151 Console.WriteLine();
152 // Print the row separator border
153 Console.WriteLine(" +");
154 for (int col = 0; col < board.Size; col++)
155 {
156 Console.WriteLine("-----");
157 }
158 Console.WriteLine();
159 }
160 }
161 }
162 }
163 }
164 }
165 }
166 }
167 }
168 }
169 }
170 }
171 }
172 }
173 }
174 }
175 }
176 }
177 }
178 }
179 }
180 }
181 }
182 }
183 }
184 }
185 }
186 }
187 }
188 }
189 }
190 }
191 }
192 }
193 }
194 }
195 }
196 }
197 }
198 }
199 }
200 }
201 }
202 }
203 }
204 }
205 }
206 }

```

Figure 25: The print board method in the utility class

```

170 // Summary
171 // Get the row and column for the piece
172 // Summary
173 // Returns
174 // Returns
175 internal static Tuple<int, int> GetRowAndCol()
176 {
177 // Declare and initialize
178 int row = -1;
179 int col = -1;
180
181 // Get the row from the user
182 Console.WriteLine("Enter the row number of the piece: ");
183 // wait for the correct response
184 while (!int.TryParse(Console.ReadLine(), out row) || row >= 0 || row <= 7)
185 {
186 // Display user message if input is invalid
187 Console.ForegroundColor = ConsoleColor.Red;
188 Console.WriteLine("Invalid input. Please enter a number between 0 and 7: ");
189 Console.ResetColor();
190 }
191
192 // Get the column from the user
193 Console.WriteLine("Enter the column number of the piece: ");
194 // wait for the correct response
195 while (!int.TryParse(Console.ReadLine(), out col) || col >= 0 || col <= 7)
196 {
197 // Display user message if input is invalid
198 Console.ForegroundColor = ConsoleColor.Red;
199 Console.WriteLine("Invalid input. Please enter a number between 0 and 7: ");
200 Console.ResetColor();
201 }
202
203 // Return the data
204 return Tuple.Create(row, col);
205 }
206 }

```

Figure 26: The get row and column method in the utility class

```

Program.cs - x
ChessBoard\ConsoleApp - Utility - (G) GetRowAndCol

16 //-----
17 // Start of Main Method
18 //-----
19
20 // Declare and Initialize
21 string pieceType = "";
22 string pieceColor = "";
23 PieceType resultType = PieceType.None;
24 PieceColor resultColor = PieceColor.None;
25 Tuple<int, int> result;
26 BoardLogic boardLogic = new BoardLogic();
27
28 // Print a welcome message for the user
29 Console.WriteLine("Hello, Chess Players!");
30
31 // Create a new chess board
32 BoardModel board = new BoardModel(0);
33
34 // Show the empty board
35 Utility.PrintBoard(board);
36
37 // Place a few preexisting pieces on the board to demonstrate the effectiveness
38 // of the MarkLegalMoves and the subsequent MarkValid moves functions for the various
39 // pieces. I played around with this a bit for a while making sure all the pieces recorded
40 // correct legal moves.
41 board.Grid[2, 5].PieceOccupyingCell = new ChessPiece(PieceType.Rook, PieceColor.Black);
42 board.Grid[5, 2].PieceOccupyingCell = new ChessPiece(PieceType.King, PieceColor.Black);
43 board.Grid[6, 3].PieceOccupyingCell = new ChessPiece(PieceType.Bishop, PieceColor.White);
44 board.Grid[1, 0].PieceOccupyingCell = new ChessPiece(PieceType.Queen, PieceColor.White);
45 board.Grid[5, 0].PieceOccupyingCell = new ChessPiece(PieceType.Pawn, PieceColor.White);
46 board.Grid[2, 7].PieceOccupyingCell = new ChessPiece(PieceType.Knight, PieceColor.Black);
47 // Print pre filled board
48 Utility.PrintBoard(board);
49
50 // Prompt the user for the type of chess piece
51 Console.WriteLine("Enter the type of piece you want to place (Pawn, Knight, Rook, Bishop, Queen, or King): ");
52 // wait for a correct response while storing results in pieceType and resultType. The addition of int try parse is to
53 // force errors in the entry value when integers are entered because in enums an int is a valid entry.
54 // (PieceType.Pawn == 1) will evaluate to true so I need to prevent integers from being entered on these lines.
55 // out simply discards the out value
56 while (int.TryParse(pieceType, out _) || (resultType = PieceTypeConverter.ConvertStringToPieceType(pieceType)) == PieceType.None)
57 {
58     // Display user message if input is invalid
59     Console.ForegroundColor = ConsoleColor.Red;
60     Console.WriteLine($"Invalid input. {pieceType} is not a valid piece type. \nPlease enter a Valid chess piece type: ");
61     Console.ResetColor();
62 }
63
64 Console.WriteLine("Enter the color (Black or White): ");
65 while (int.TryParse(pieceColor, out _) || (resultColor = PieceColorConverter.ConvertStringToPieceColor(pieceColor)) == PieceColor.None)
66 {
67     // Display user message if input is invalid
68     Console.ForegroundColor = ConsoleColor.Red;
69     Console.WriteLine($"Invalid input. {pieceColor} is not a valid piece color. \nPlease enter a Valid chess piece color: ");
70     Console.ResetColor();
71 }
72
73 // Prompt the user for the location of the chess piece
74 result = Utility.GetRowAndCol();
75
76 // Mark the legal moves based on the input
77 board = boardLogic.MarkLegalMoves(board, board.Grid[result.Item1, result.Item2], resultType, resultColor);
78
79 // Print out the new chess board
80 Utility.PrintBoard(board);
81
82 //-----
83 // End of Main Method
84 //-----
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100

```

Figure 27: The main method for the console chess app

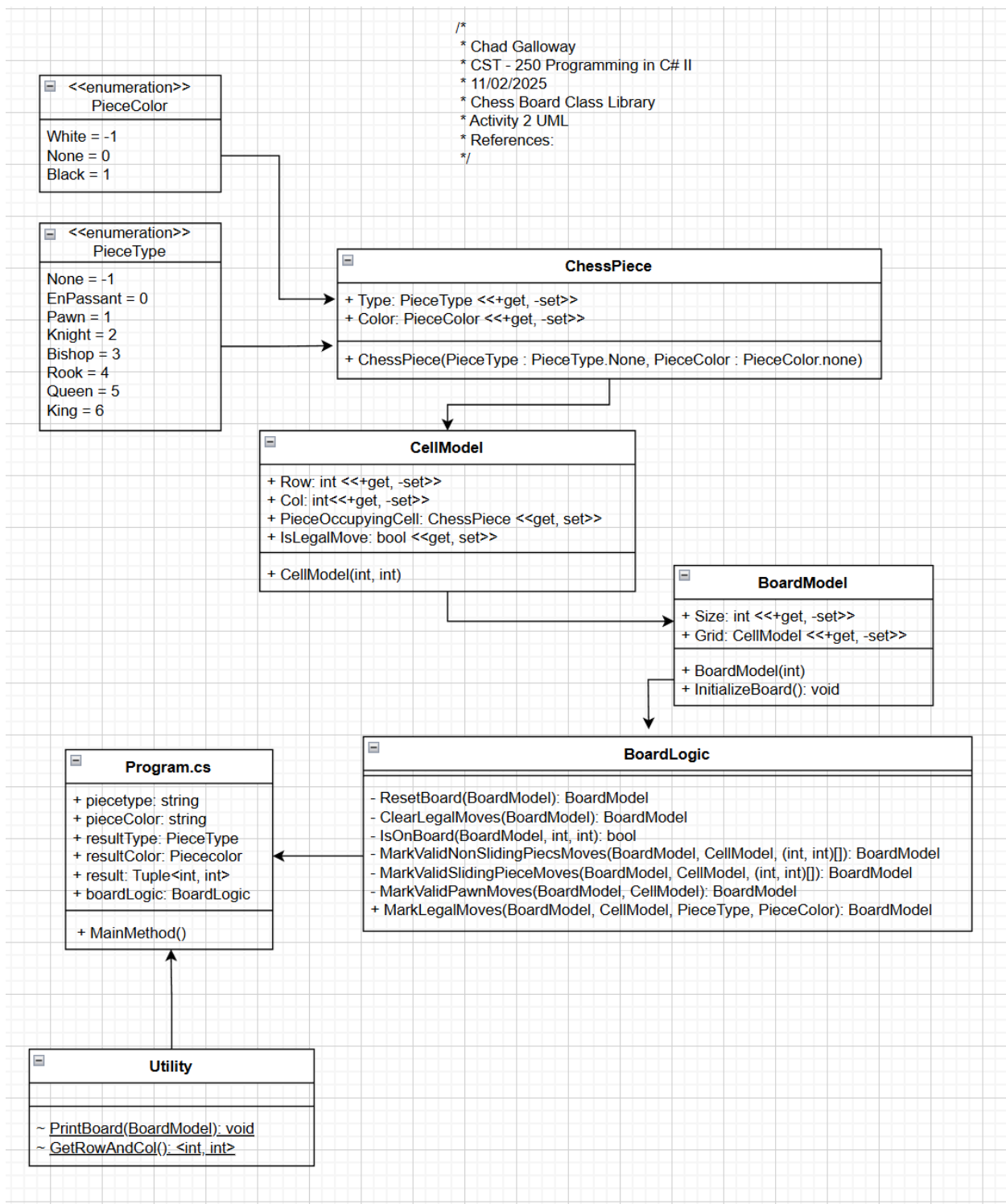


Figure 28: Current state of the application's UML

Chess Board

Select a chess piece and its location on the board and see the legal moves

Pieces:

0, 0	1, 0	2, 0	3, 0	4, 0	5, 0	6, 0	7, 0
0, 1	1, 1	2, 1	3, 1	4, 1	5, 1	6, 1	7, 1
0, 2	1, 2	2, 2	3, 2	4, 2	5, 2	6, 2	7, 2
0, 3	1, 3	2, 3	3, 3	4, 3	5, 3	6, 3	7, 3
0, 4	1, 4	2, 4	3, 4	4, 4	5, 4	6, 4	7, 4
0, 5	1, 5	2, 5	3, 5	4, 5	5, 5	6, 5	7, 5
0, 6	1, 6	2, 6	3, 6	4, 6	5, 6	6, 6	7, 6
0, 7	1, 7	2, 7	3, 7	4, 7	5, 7	6, 7	7, 7

Figure 29: The Windows Forms app running

```

1  //
2  //
3  //
4  //
5  //
6  //
7  //
8  //
9  //
10 //
11 //
12 //
13 //
14 //
15 //
16 //
17 //
18 //
19 //
20 //
21 //
22 //
23 //
24 //
25 //
26 //
27 //
28 //
29 //
30 //
31 //
32 //
33 //
34 //
35 //
36 //
37 //
38 //
39 //
40 //
41 //
42 //
43 //
44 //
45 //
46 //
47 //
48 //
49 //
50 //
51 //
52 //
53 //
54 //
55 //
56 //
57 //
58 //
59 //
60 //
61 //
62 //
63 //
64 //
65 //
66 //
67 //
68 //
69 //
70 //
71 //
72 //
73 //
74 //
75 //
76 //
77 //
78 //
79 //
80 //
81 //
82 //
83 //
84 //
85 //
86 //
87 //
88 //
89 //
90 //
91 //
92 //
93 //

```

Figure 30: FrmChessBoard.cs citations and constructor.

```

39 //
40 //
41 //
42 //
43 //
44 //
45 //
46 //
47 //
48 //
49 //
50 //
51 //
52 //
53 //
54 //
55 //
56 //
57 //
58 //
59 //
60 //
61 //
62 //
63 //
64 //
65 //
66 //
67 //
68 //
69 //
70 //
71 //
72 //
73 //
74 //
75 //
76 //
77 //
78 //
79 //
80 //
81 //
82 //
83 //
84 //
85 //
86 //
87 //
88 //
89 //
90 //
91 //
92 //

```

Figure 31: FrmChessBoard.cs SetUpButtons() method.

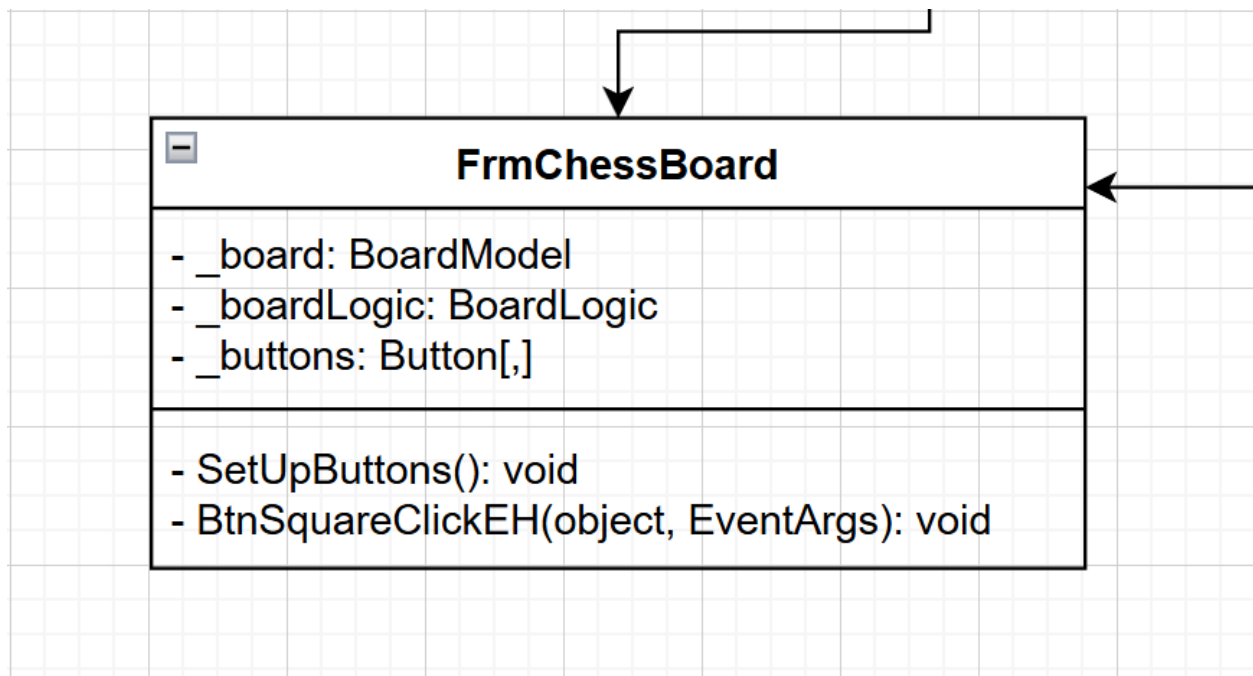


Figure 32: UML for the FrmChessBoard application.

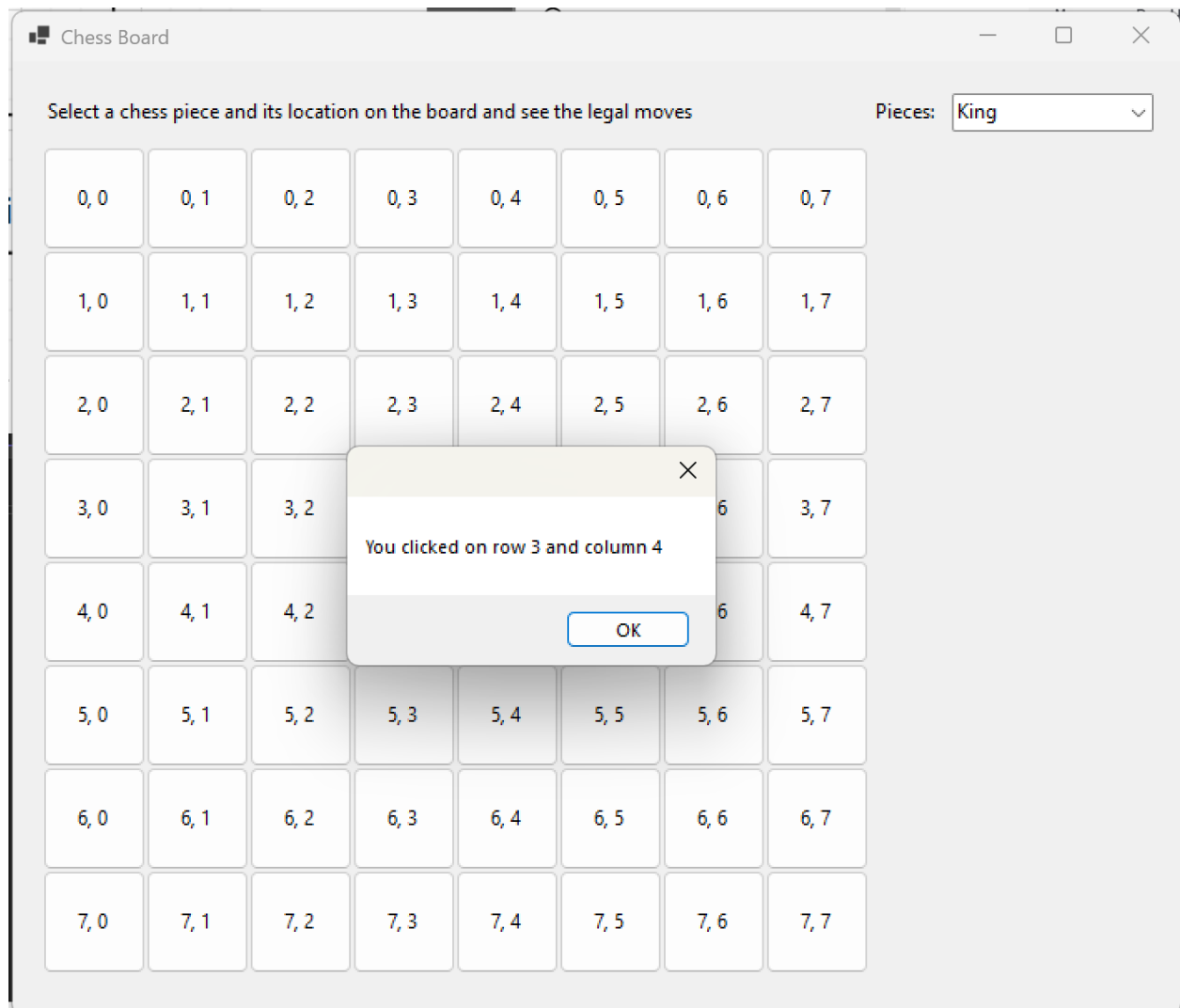


Figure 33: Message box

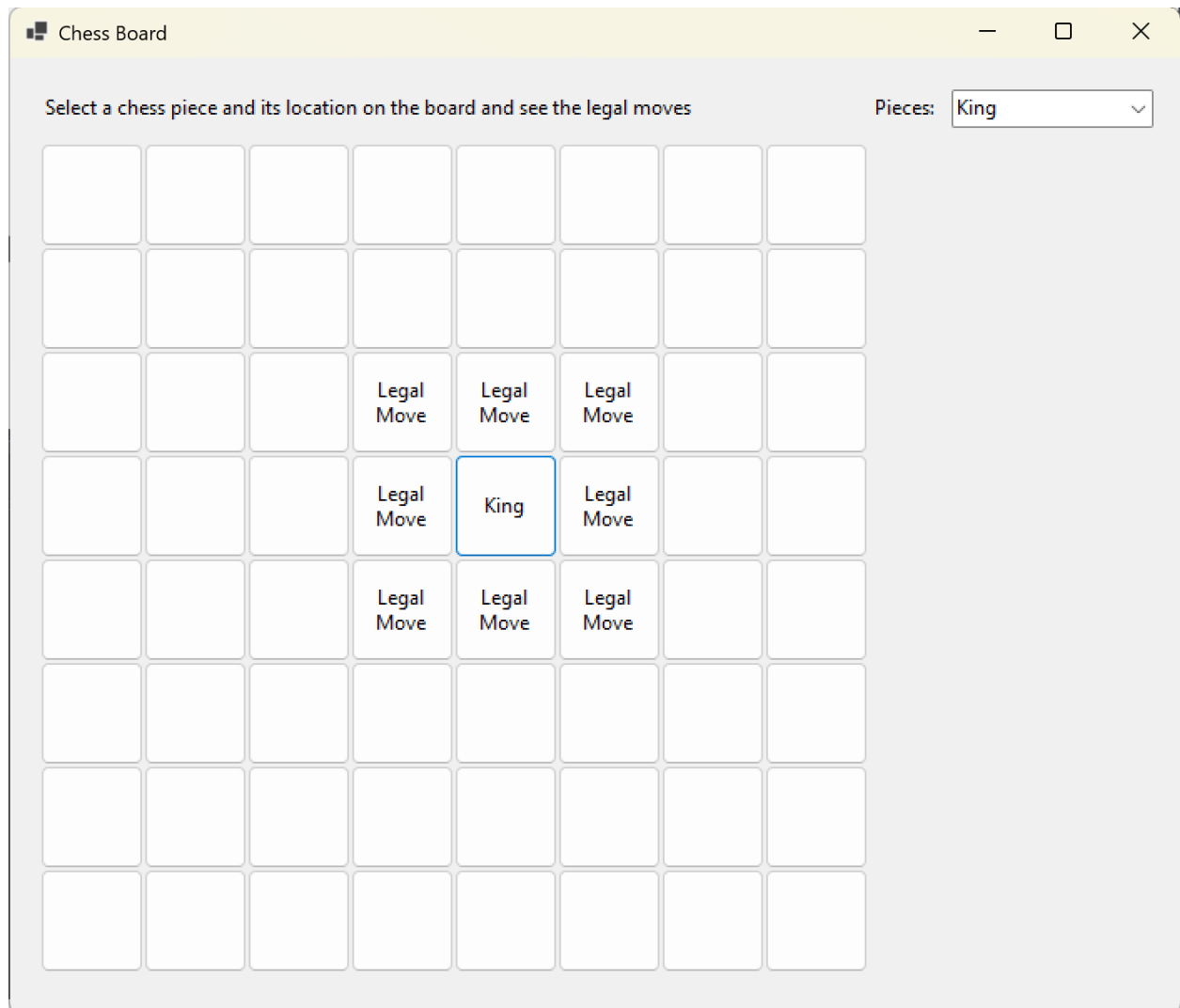


Figure 34: King

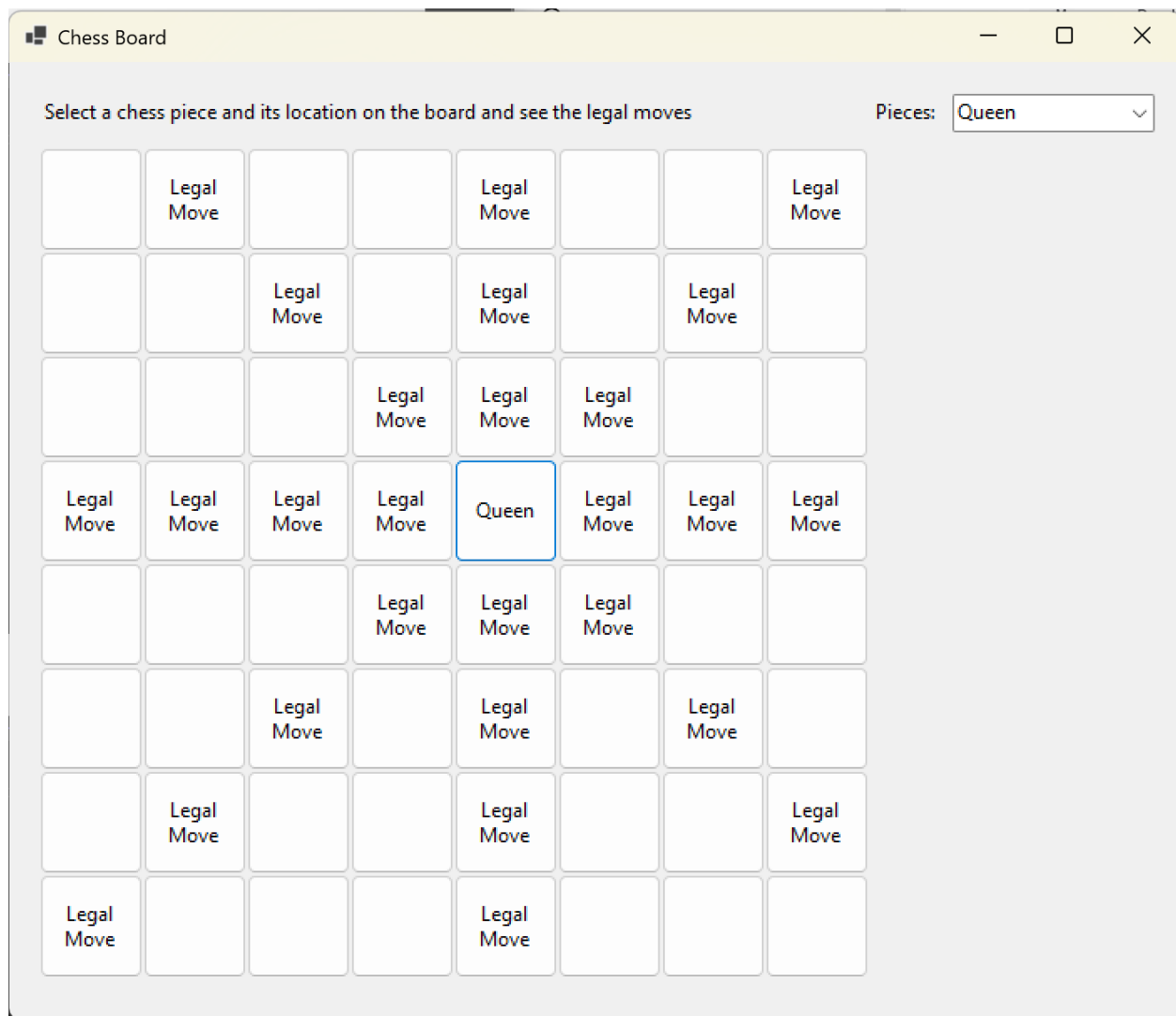


Figure 35: Queen

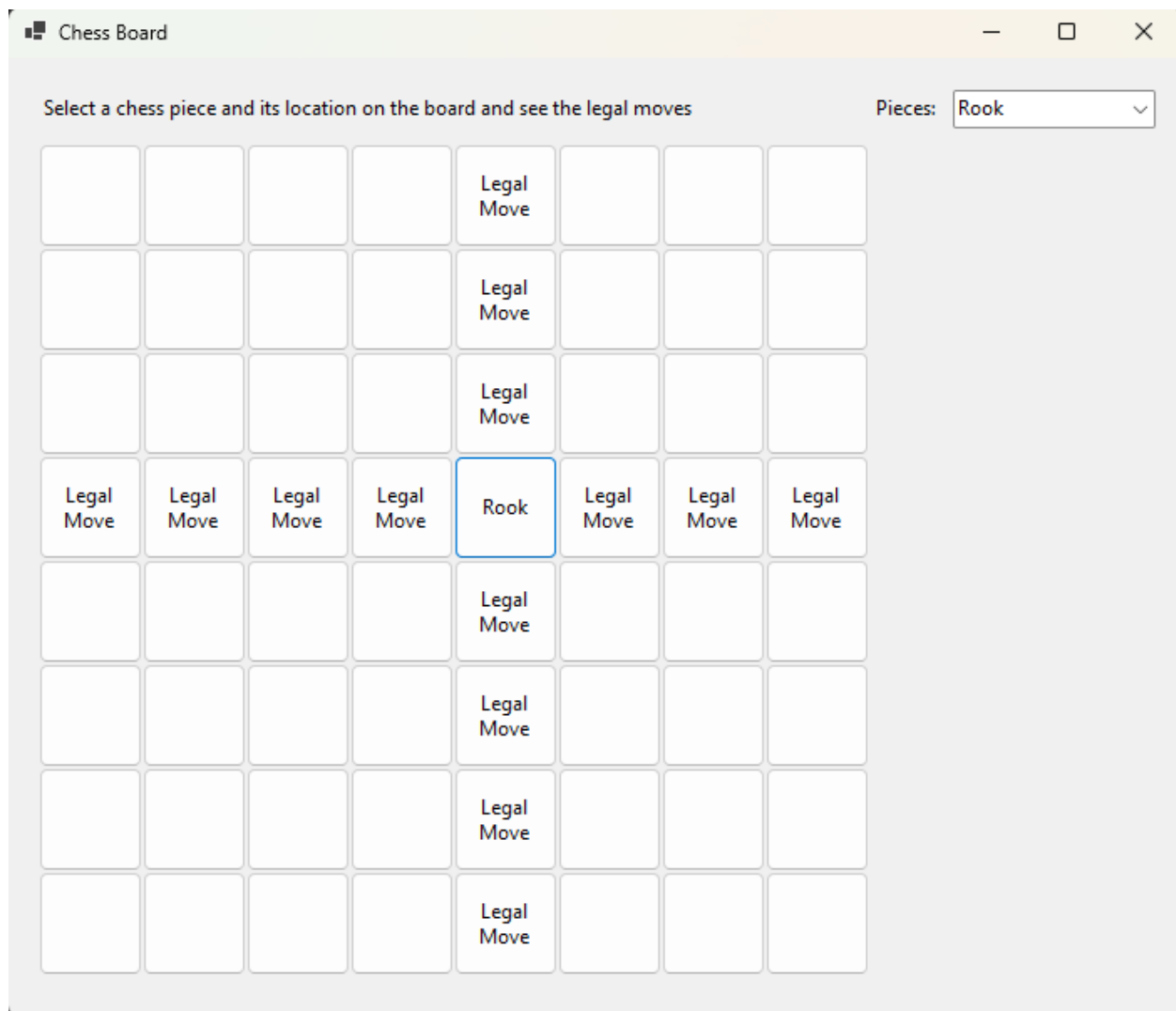


Figure 36: Rook

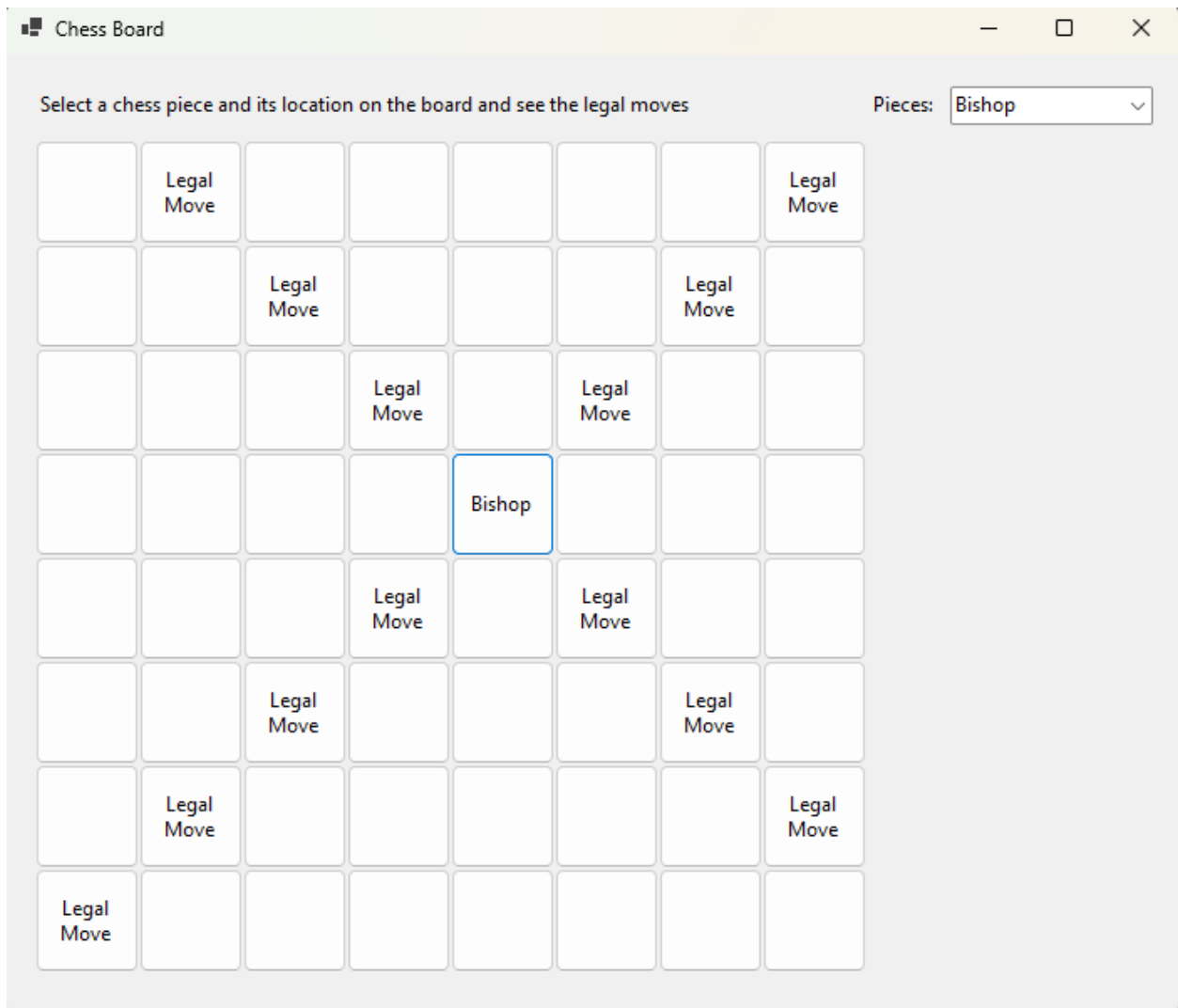


Figure 37: Bishop

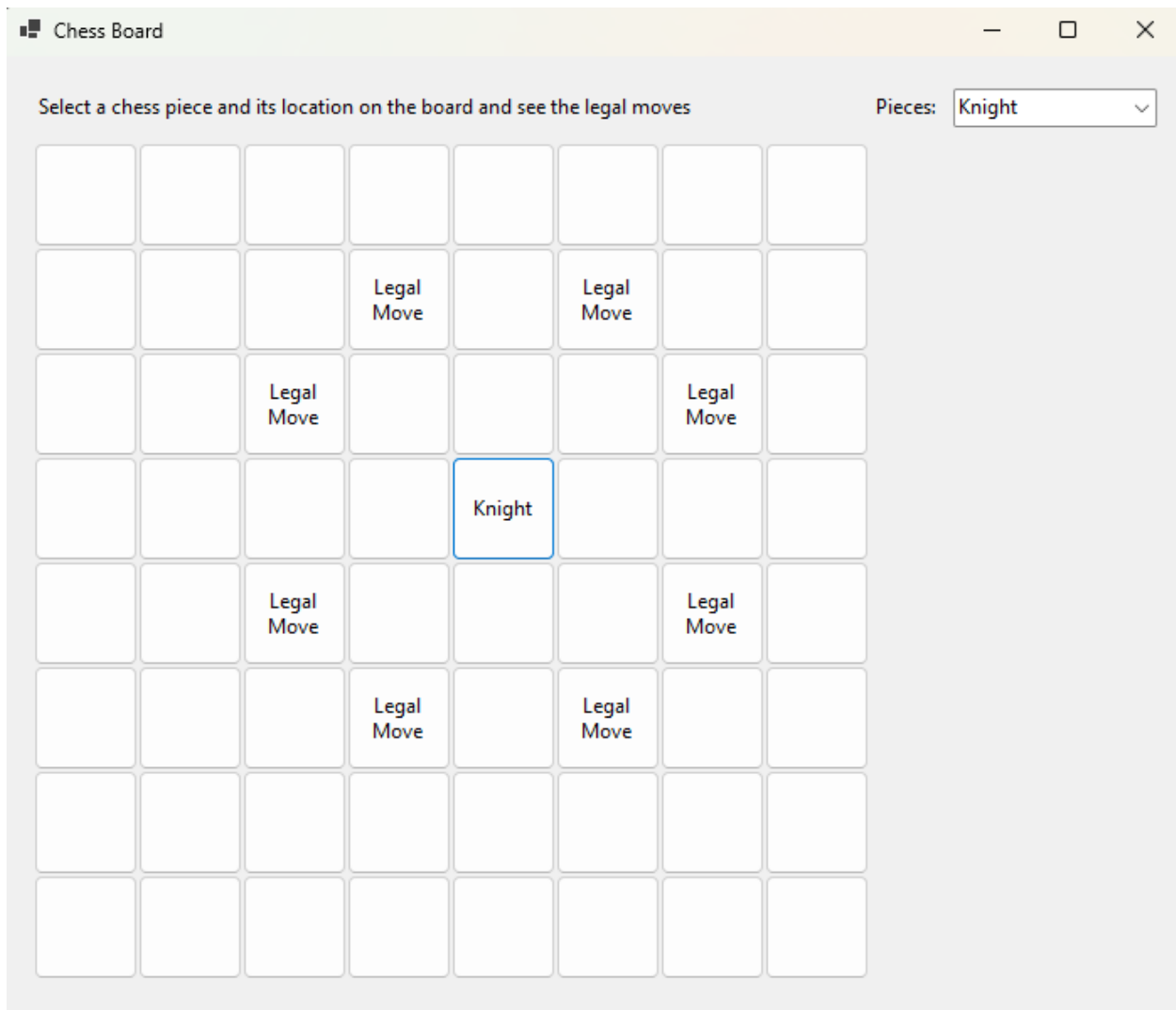


Figure 38: Knight

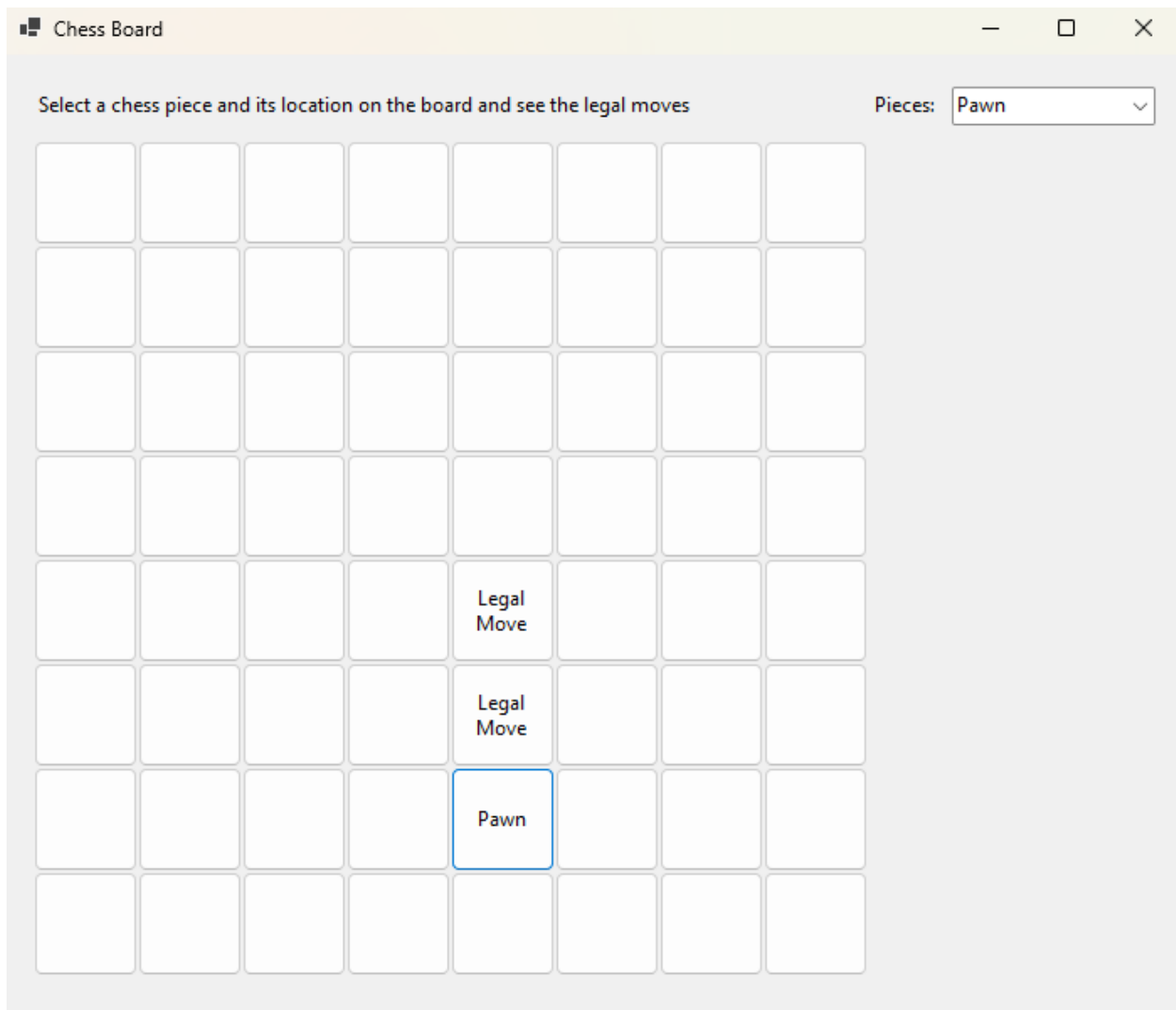


Figure 39: Pawn

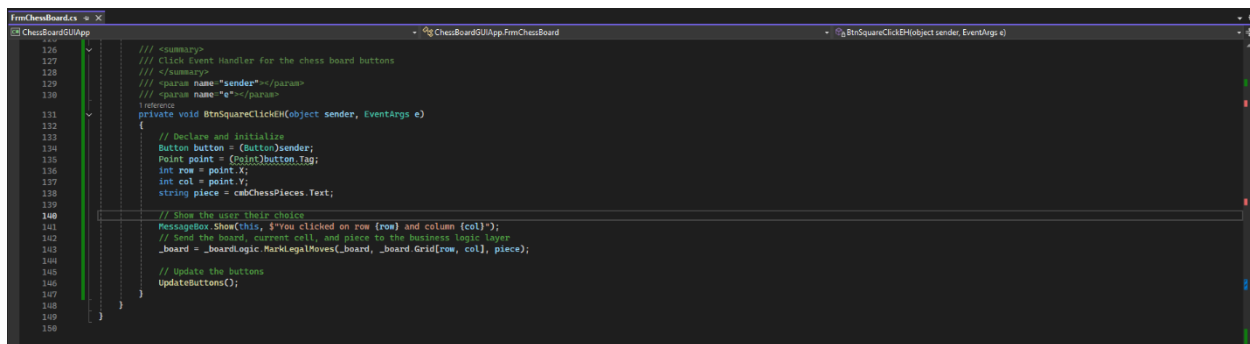


Figure 40: BtnSquareClickEH

```

114 115 116 117 118 119 120 121 122 123 124 125 126 127 128 129 130 131 132 133 134 135 136 137 138 139 140 141 142 143 144 145 146 147 148 149 150 151 152 153 154 155 156 157 158 159 160 161 162 163 164 165 166 167 168 169 170 171 172 173 174 175 176 177 178 179 180 181 182 183 184 185 186 187 188 189 190 191 192 193 194 195 196 197 198 199 200 201 202 203 204 205 206 207 208 209 210 211 212 213 214 215 216 217 218 219 220 221 222 223 224 225 226 227 228 229 230 231 232 233 234 235 236 237 238 239 240 241 242 243 244 245 246 247 248 249 250 251 252 253 254 255 256 257 258 259 260 261 262 263 264 265 266 267 268 269 270 271 272 273 274 275 276 277 278 279 280 281 282 283 284 285 286 287 288 289 290 291 292 293 294 295 296 297 298 299 300 301 302 303 304 305 306 307 308 309 310 311 312 313 314 315 316 317 318 319 320 321 322 323 324 325 326 327 328 329 330 331 332 333 334 335 336 337 338 339 340 341 342 343 344 345 346 347 348 349 350 351 352 353 354 355 356 357 358 359 360 361 362 363 364 365 366 367 368 369 370 371 372 373 374 375 376 377 378 379 380 381 382 383 384 385 386 387 388 389 390 391 392 393 394 395 396 397 398 399 400 401 402 403 404 405 406 407 408 409 410 411 412 413 414 415 416 417 418 419 420 421 422 423 424 425 426 427 428 429 430 431 432 433 434 435 436 437 438 439 440 441 442 443 444 445 446 447 448 449 450 451 452 453 454 455 456 457 458 459 460 461 462 463 464 465 466 467 468 469 470 471 472 473 474 475 476 477 478 479 480 481 482 483 484 485 486 487 488 489 490 491 492 493 494 495 496 497 498 499 500 501 502 503 504 505 506 507 508 509 510 511 512 513 514 515 516 517 518 519 520 521 522 523 524 525 526 527 528 529 530 531 532 533 534 535 536 537 538 539 540 541 542 543 544 545 546 547 548 549 550 551 552 553 554 555 556 557 558 559 560 561 562 563 564 565 566 567 568 569 570 571 572 573 574 575 576 577 578 579 580 581 582 583 584 585 586 587 588 589 590 591 592 593 594 595 596 597 598 599 600 601 602 603 604 605 606 607 608 609 610 611 612 613 614 615 616 617 618 619 620 621 622 623 624 625 626 627 628 629 630 631 632 633 634 635 636 637 638 639 640 641 642 643 644 645 646 647 648 649 650 651 652 653 654 655 656 657 658 659 660 661 662 663 664 665 666 667 668 669 670 671 672 673 674 675 676 677 678 679 680 681 682 683 684 685 686 687 688 689 690 691 692 693 694 695 696 697 698 699 700 701 702 703 704 705 706 707 708 709 710 711 712 713 714 715 716 717 718 719 720 721 722 723 724 725 726 727 728 729 730 731 732 733 734 735 736 737 738 739 740 741 742 743 744 745 746 747 748 749 750 751 752 753 754 755 756 757 758 759 760 761 762 763 764 765 766 767 768 769 770 771 772 773 774 775 776 777 778 779 780 781 782 783 784 785 786 787 788 789 790 791 792 793 794 795 796 797 798 799 800 801 802 803 804 805 806 807 808 809 810 811 812 813 814 815 816 817 818 819 820 821 822 823 824 825 826 827 828 829 830 831 832 833 834 835 836 837 838 839 840 841 842 843 844 845 846 847 848 849 850 851 852 853 854 855 856 857 858 859 860 861 862 863 864 865 866 867 868 869 870 871 872 873 874 875 876 877 878 879 880 881 882 883 884 885 886 887 888 889 890 891 892 893 894 895 896 897 898 899 900 901 902 903 904 905 906 907 908 909 910 911 912 913 914 915 916 917 918 919 920 921 922 923 924 925 926 927 928 929 930 931 932 933 934 935 936 937 938 939 940 941 942 943 944 945 946 947 948 949 950 951 952 953 954 955 956 957 958 959 960 961 962 963 964 965 966 967 968 969 970 971 972 973 974 975 976 977 978 979 980 981 982 983 984 985 986 987 988 989 990 991 992 993 994 995 996 997 998 999 1000

```

Figure 41: Update Buttons

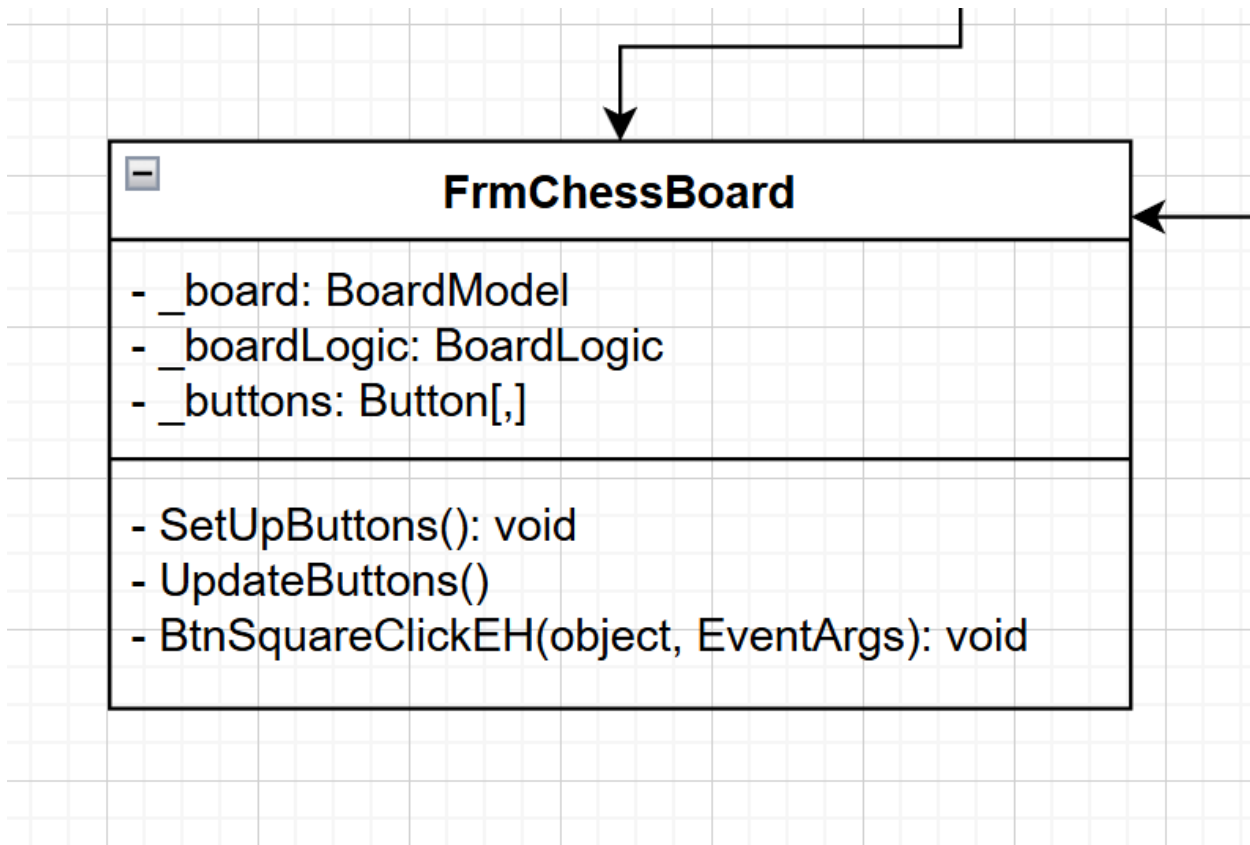


Figure 42: UML

Challenges

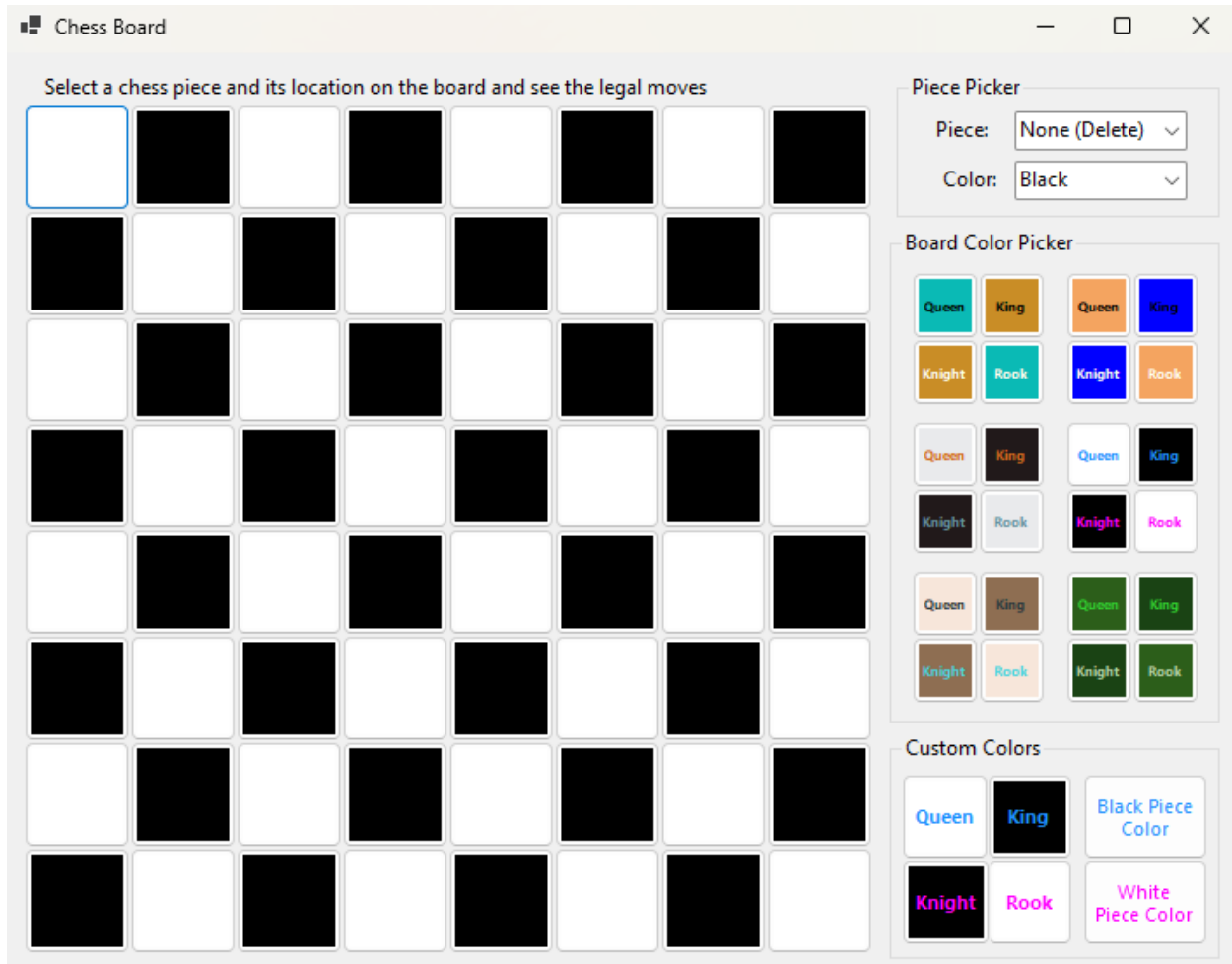


Figure 43: Default load state of the application

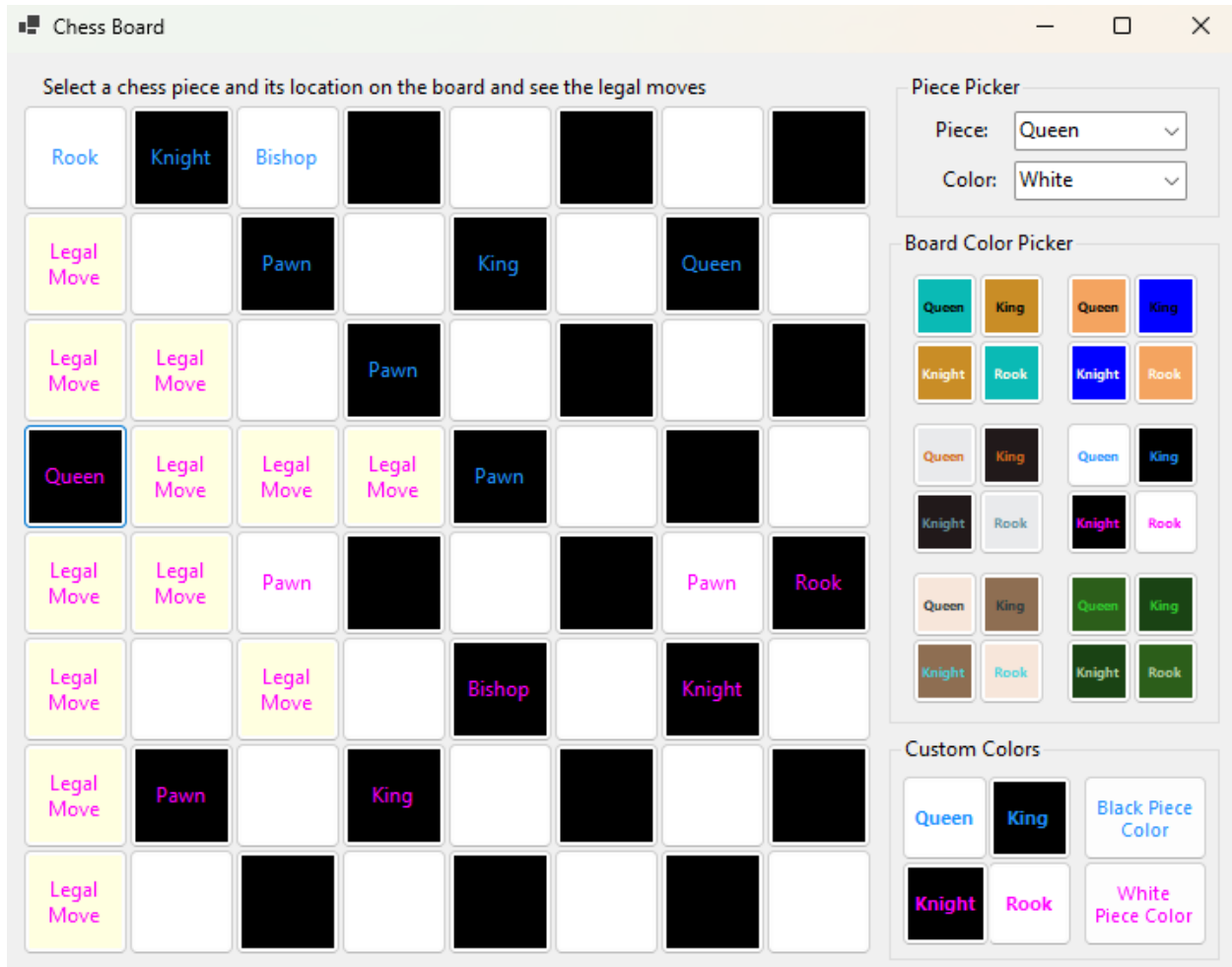


Figure 44: A few pieces placed on the board. I made the decision to not clear pieces off the board when putting a new piece.

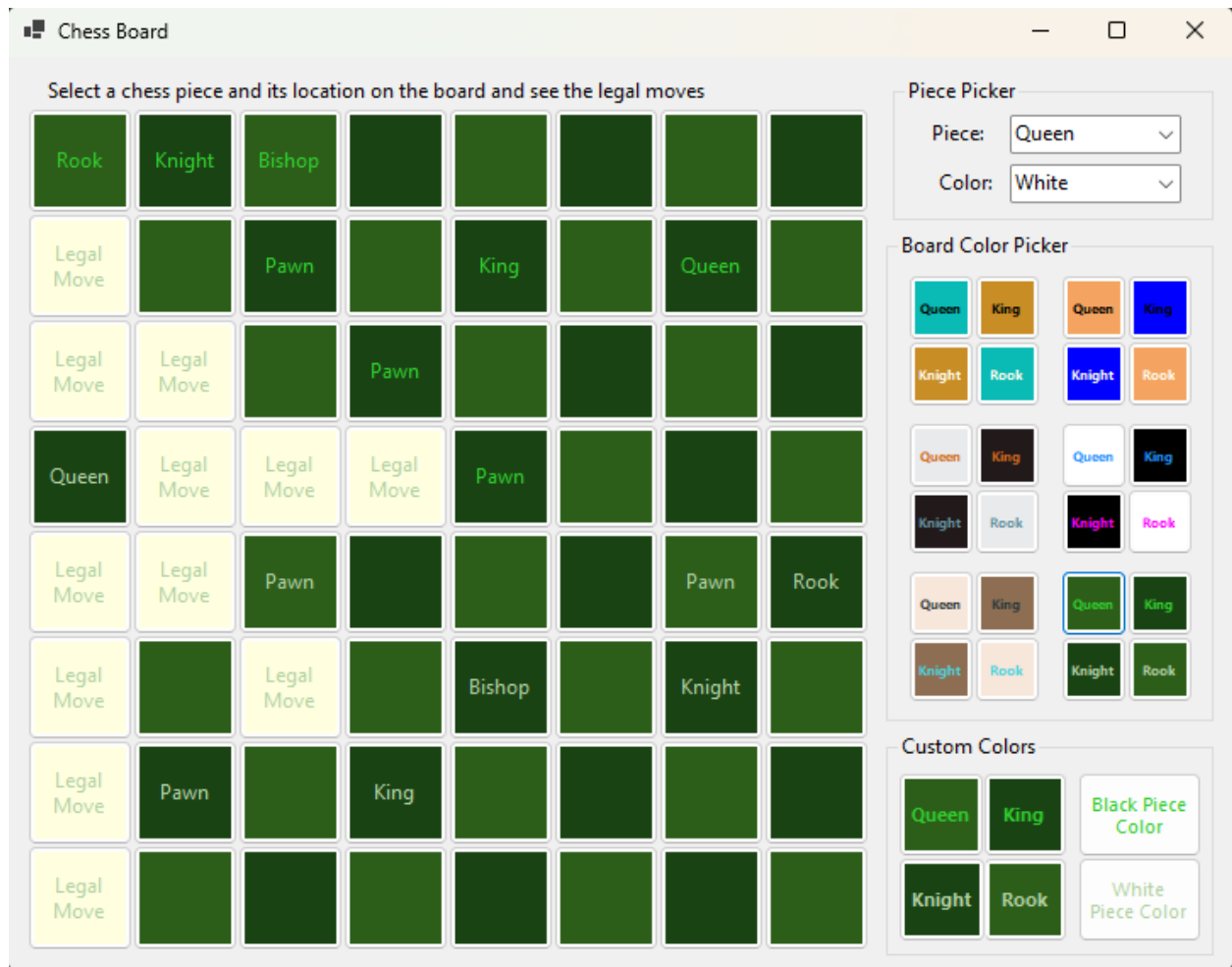


Figure 45: Changed color of the board by clicking one of the preset color palettes

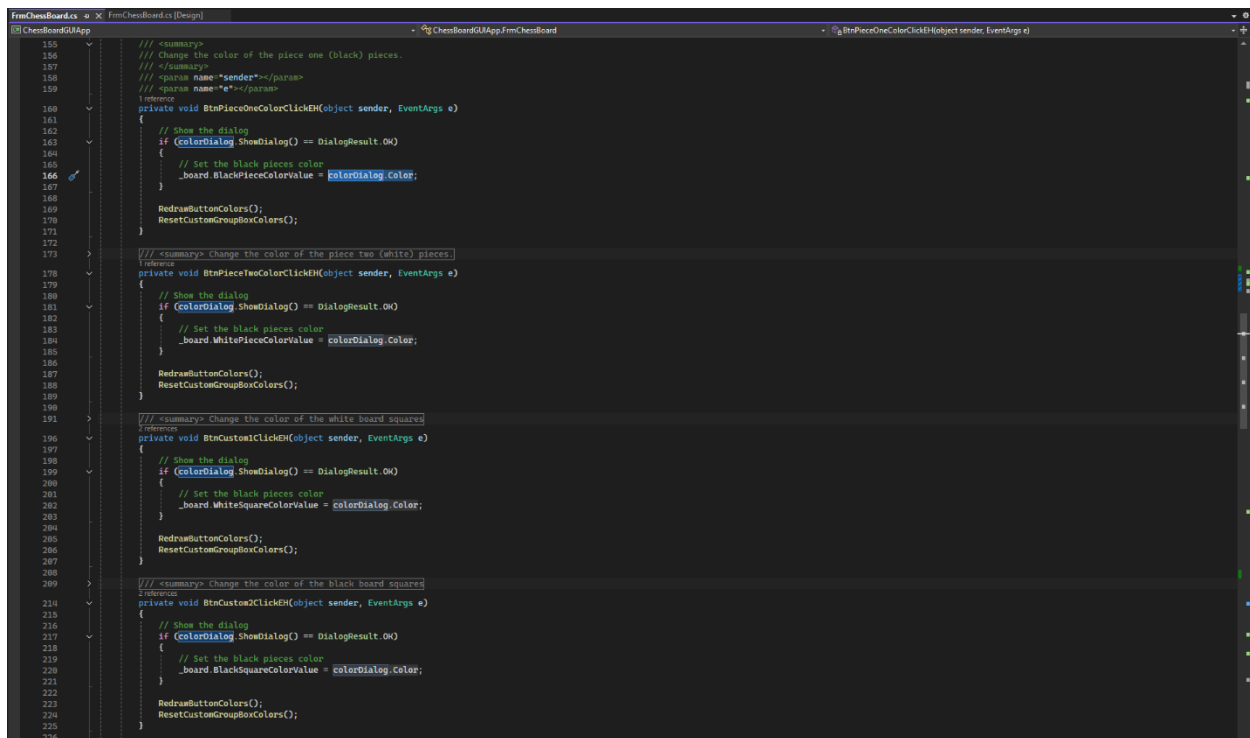


Figure 46: Custom color picker methods

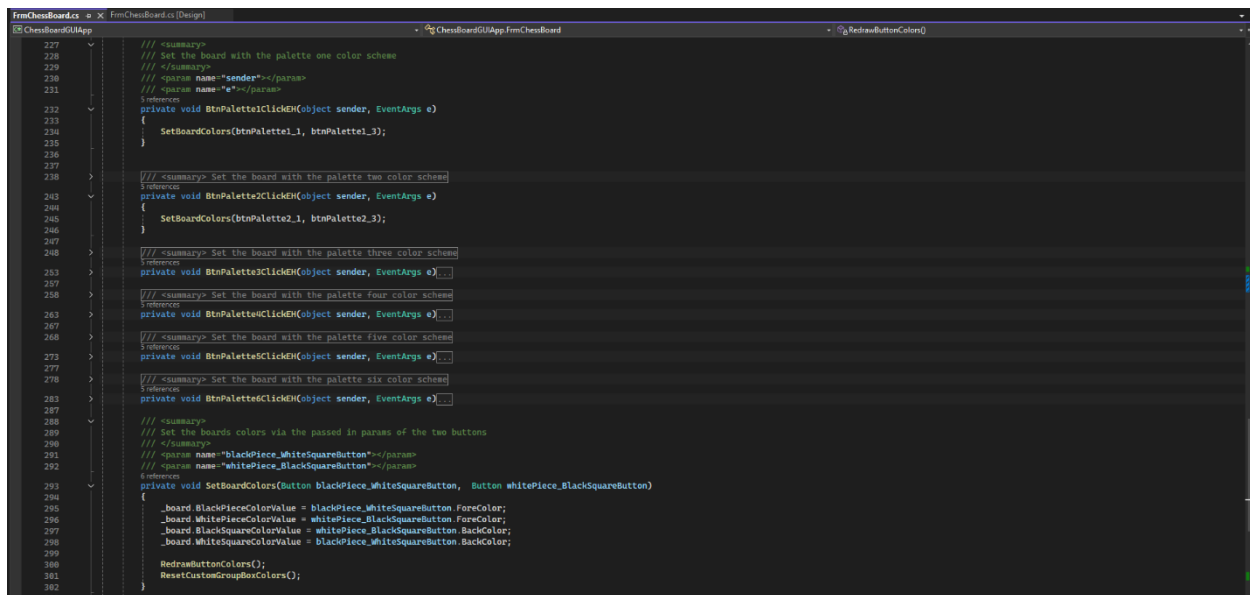


Figure 47: Group of Methods for the preset color palettes

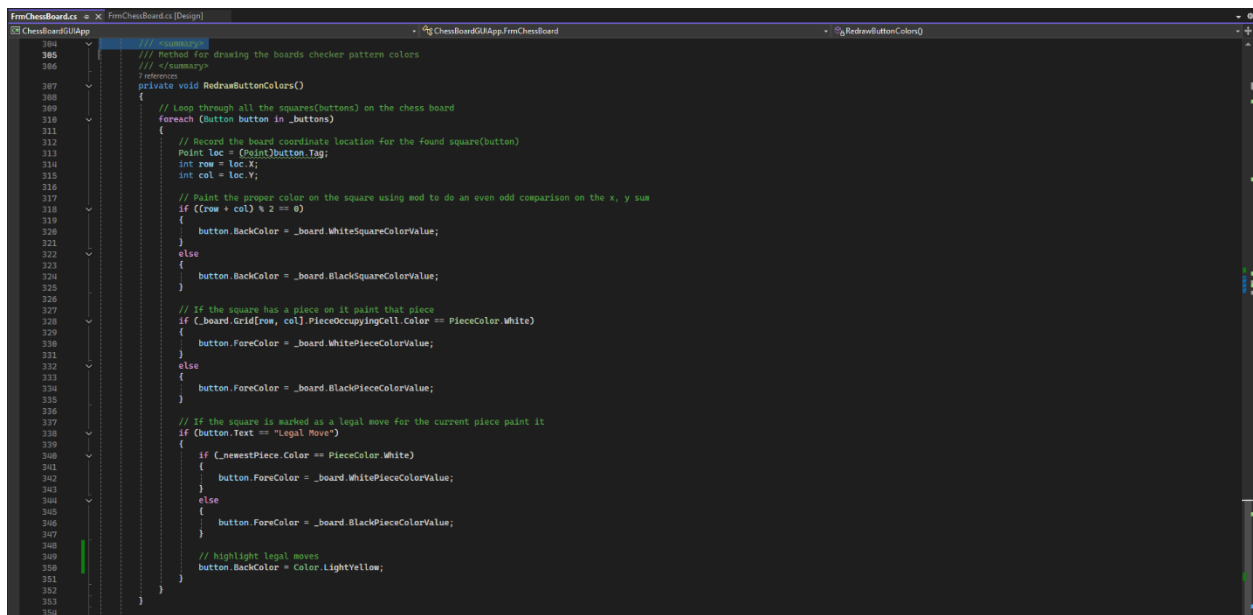


Figure 48: `RedrawButtonColors()` for redrawing the buttons inside the chess board when needed.

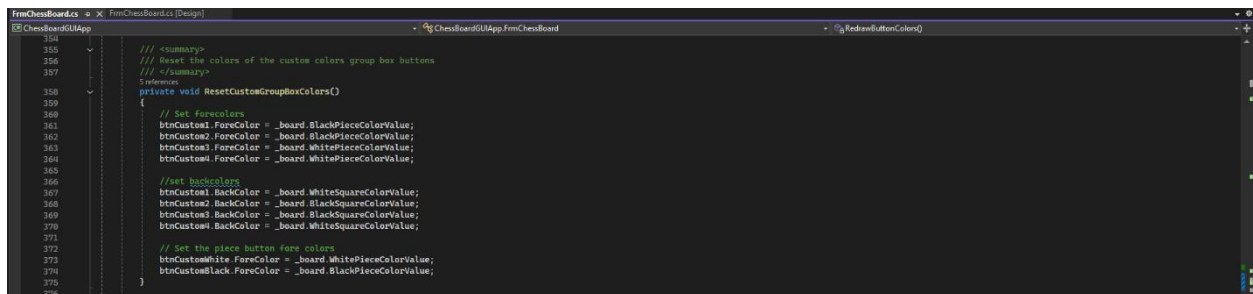


Figure 49: When we change any color, we need to update the custom color group box items. That is handled here `ResetCustomGroupBoxColors()`;

```

frmChessBoard.cs  frmChessBoard.cs [Design]  - ChessBoardGUIApp.FrmChessBoard  - RedrawButtonColors()
ChessBoardGUIApp
277  /// <summary>
278  /// Method for catching mouse entry on a panels buttons so we can paint a border on the panel the button is in
279  /// </summary>
280  /// <param name="sender"></param>
281  /// <param name="e"></param>
282  24 references
382  private void BtnPaletteMouseEnter(object sender, EventArgs e)
383  {
384      ((Control)sender).Parent.Invalidate(); // Force panel repaint
385      _panelHighlight = true;
386  }
387
388  /// <summary>
389  /// Method for catching mouse exit on a panels buttons so we can remove the panels border
390  /// </summary>
391  /// <param name="sender"></param>
392  /// <param name="e"></param>
393  24 references
394  private void BtnPaletteMouseLeave(object sender, EventArgs e)
395  {
396      ((Control)sender).Parent.Invalidate(); // Force panel repaint
397      _panelHighlight = false;
398  }
399
400  /// <summary>
401  /// Method to determine if a panel boarder is to be applied and apply it.
402  /// </summary>
403  /// <param name="sender"></param>
404  /// <param name="e"></param>
405  4 references
406  private void PnlPalettePaint(object sender, PaintEventArgs e)
407  {
408      var panel = (Panel)sender;
409      // If flagged for highlighting do it.
410      if (_panelHighlight)
411      {
412          using (Pen p = new Pen(Color.DarkCyan, 2))
413          {
414              e.Graphics.DrawRectangle(p, 0, 0, panel.Width - 1, panel.Height - 1);
415          }
416      }
417  }
418

```

Figure 50: Methods for handling the mouse over highlighting of the preset color palette panels

Week 3 FLOW CHART

```
/*  
 * Chad Galloway  
 * CST - 250 Programming in C# II  
 * 11/02/2025  
 * Count to One  
 * Activity 2 Part 3i  
 * References:  
 */
```

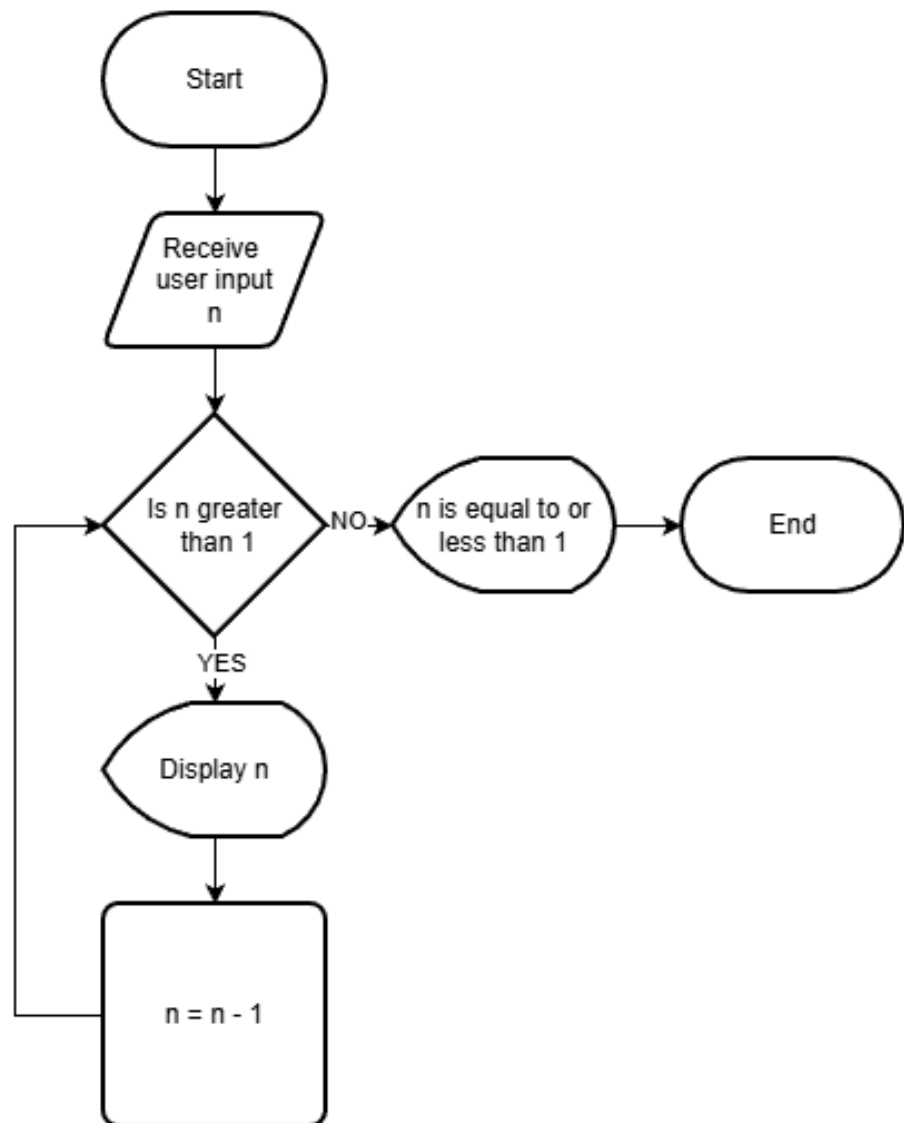


Figure 51: Flow chart of Activity 2 Part 3 i Count to One

/*
* Chad Galloway
* CST - 250 Programming in C# II
* 11/02/2025
* Factorial
* Activity 2 Part 3ii
* References:
*/

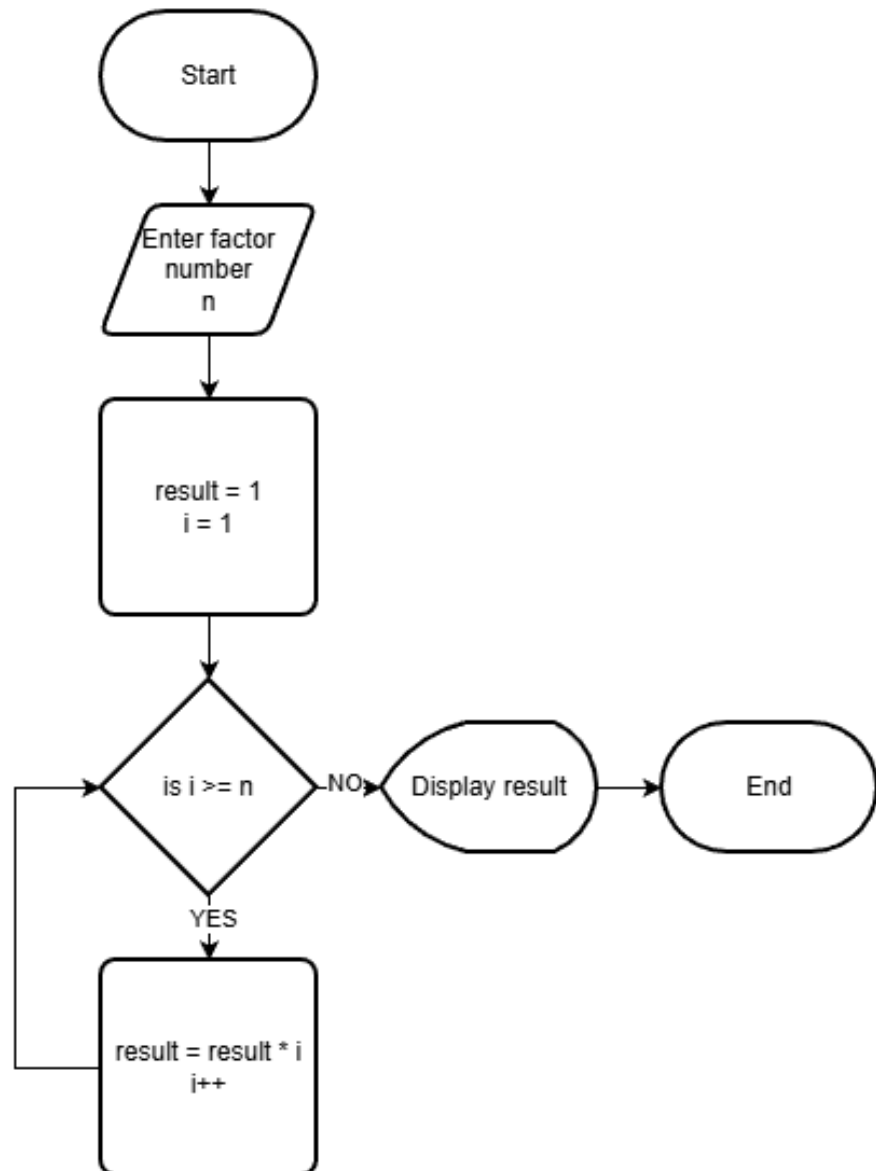


Figure 52: Flow chart of Activity 2 Part 3 ii Factorial

/*
* Chad Galloway
* CST - 250 Programming in C# II
* 11/02/2025
* Greatest Common Denominator
* Activity 2 Part 3iii
* References:
*/

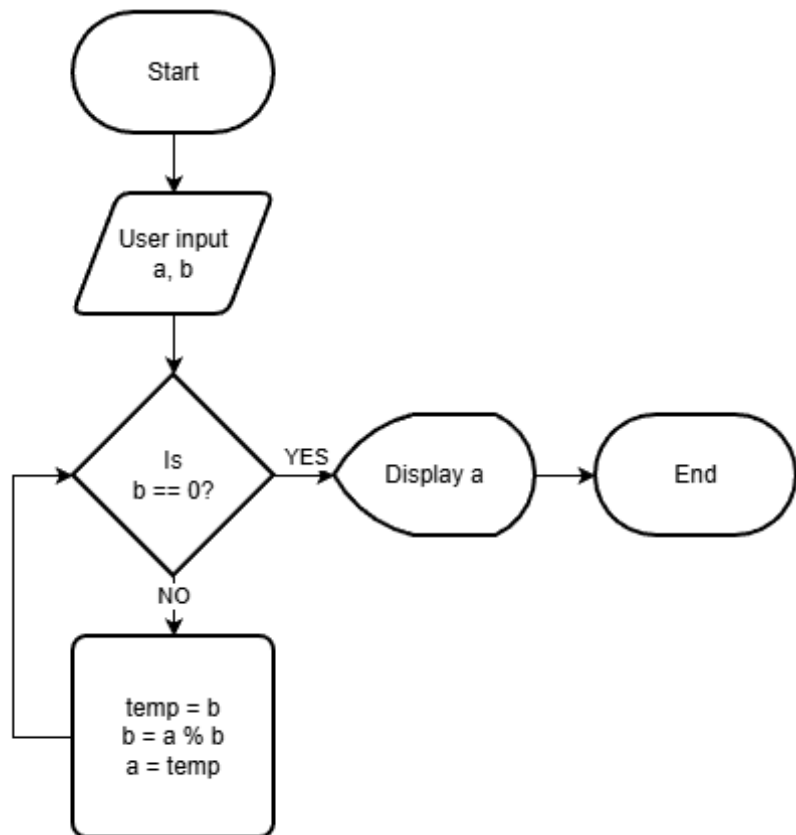


Figure 53: Flow chart of Activity 2 Part 3 iii GCD

/*
* Chad Galloway
* CST - 250 Programming in C# II
* 11/02/2025
* Flood Fill
* Activity 2 Part 3iv
* References:
*/

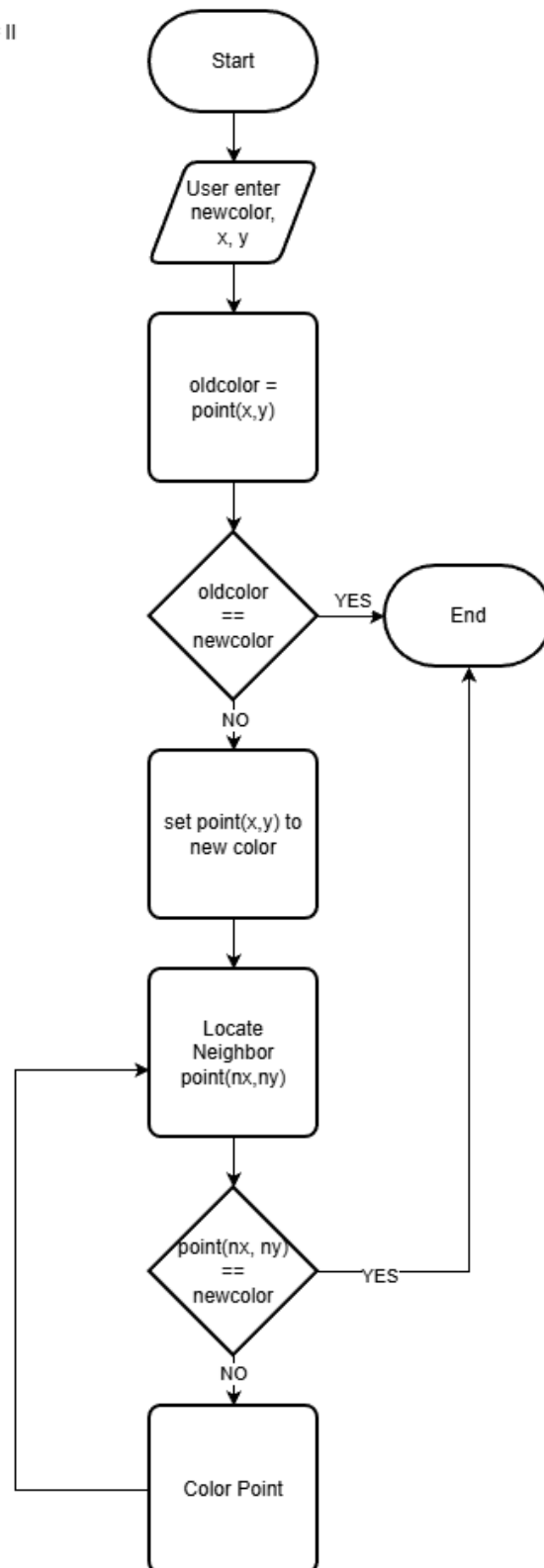


Figure 54: Flow chart of Activity 2 Part 3 iv Flood Fill

UML Class Diagram

```
/*  
* Chad Galloway  
* CST - 250 Programming in C# II  
* 11/02/2025  
* Count to One  
* Activity 2 Part 3i  
* References:  
*/
```

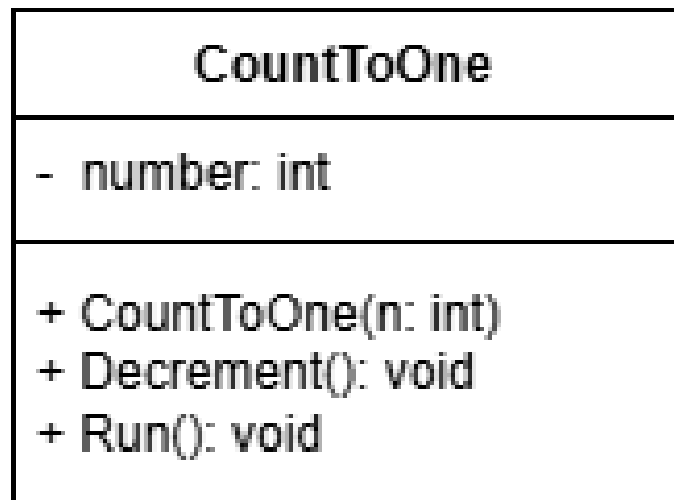


Figure 55: UML Class Diagram for Activity 2 Part 3 i Count to One

```

/*
 * Chad Galloway
 * CST - 250 Programming in C# II
 * 11/02/2025
 * Factorial
 * Activity 2 Part 3ii
 * References:
 */

```

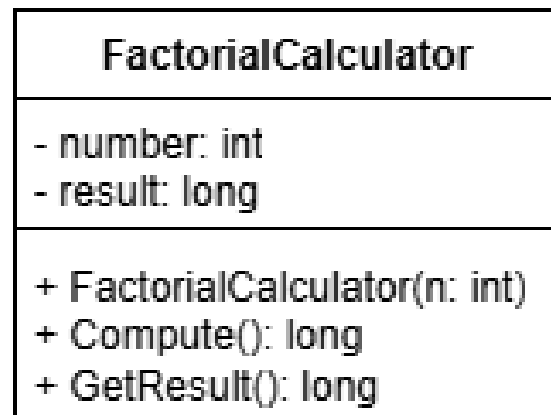


Figure 56: UML Class Diagram for Activity 2 Part 3 ii Factorial

```

/*
 * Chad Galloway
 * CST - 250 Programming in C# II
 * 11/02/2025
 * Greatest Common Denominator
 * Activity 2 Part 3iii
 * References: User. (2013, August 30). C# find the greatest common divisor. Stack Overflow.
 *             https://stackoverflow.com/questions/18541832/c-sharp-find-the-greatest-common-divisor
 */

```

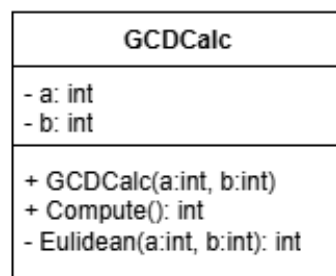


Figure 57: UML Class Diagram for Activity 2 Part 3 iii GCD

```
/*  
* Chad Galloway  
* CST - 250 Programming in C# II  
* 11/02/2025  
* Flood Fill  
* Activity 2 Part 3iv  
* References:  
*/
```

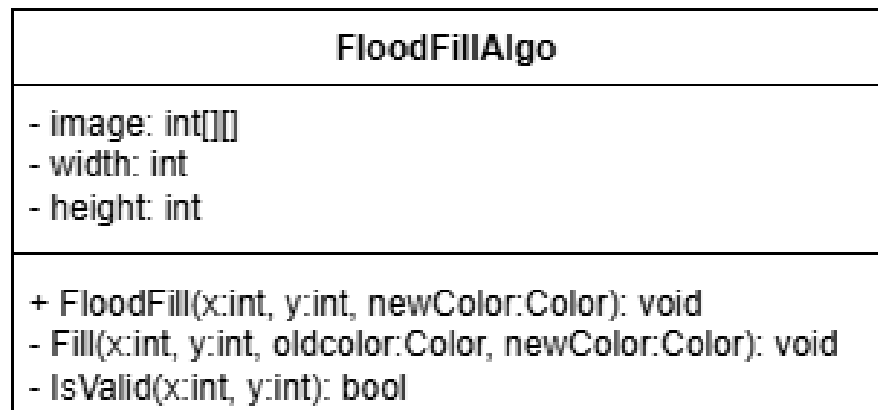


Figure 58: UML Class Diagram for Activity 2 Part 3 iv Flood Fill